Fondamenti di Programmazione

settembre e ottobre 2026

Come usare questo libro

Questo libro raccoglie le lezioni di Fondamenti di Programmazione dal 19 settembre al 29 ottobre 2026 e le rimette in un ordine che costruisce un mattone alla volta. Si parte da che cosa sono un algoritmo e uno stato, si impara la logica che serve per scrivere frasi precise (proposizioni, poi quantificatori) e la si usa infine per *dimostrare* che un programma fa quello che promette: triple di Hoare, invarianti, funzioni di terminazione, array.

È un libro leggero: pochi concetti, molti esempi eseguiti a mano. Lungo il testo trovi cinque tipi di riquadro.

- **In breve**: all'inizio di ogni capitolo, che cosa impareremo e perché.
- **Esercizio guidato**: un problema risolto passo per passo. Prova a fare ogni passo da solo prima di leggerlo.
- **Attenzione**: un errore tipico, e come evitarlo.
- **Immagine mentale**: un modo di vedere un'idea che la rende facile da ricordare.
- **Officina della dimostrazione**: in ogni capitolo un attrezzo del ragionamento (leggere una specifica, cercare un controesempio, calcolare all'indietro, trovare un invariante...); l'appendice A li riassume.

Ogni capitolo finisce con una breve **verifica**. Le risposte sono nell'appendice B.

Convenzioni. I valori di verità sono `t` (vero) e `f` (falso); T e F sono invece le *formule* sempre vera e sempre falsa. Il codice è scritto nel linguaggio didattico del corso (`:=`, `if` *:::* `then` *:::* `else` *:::* `fi`, `while` *:::* `do` *:::* `endw`); le asserzioni tra graffe non si eseguono. Le figure, gli esempi aggiuntivi, le verifiche e gli svolgimenti degli esercizi lasciati aperti nelle slide sono integrazioni didattiche, non parole del docente; dove il libro corregge un refuso delle slide lo dice.

Indice

Come usare questo libro 2

1 Dal problema al programma 4

2 Proposizioni: forma e significato 9

3 Dimostrare con le leggi 14

4 Il linguaggio del primo ordine 20

5 Verità e leggi nel primo ordine 24

6 Stati, espressioni e comandi 28

7 Triple di Hoare 32

8 Cicli: invarianti e terminazione 38

9 Array e progetto di cicli 44

Appendice A La cassetta degli attrezzi 51

Appendice B Risposte alle verifiche 52

Capitolo 1

Dal problema al programma

In breve
Al centro dell'informatica non c'è la macchina ma l'**algoritmo**: una procedura così precisa che un esecutore può seguirla senza capirla. In questo capitolo vediamo come si passa da un problema detto a parole a un programma, e scopriamo l'idea che regge tutto il corso: un calcolo è una successione di **stati**.

1.1 Che cosa studia l'informatica

L'informatica non è la tecnica dei calcolatori, né l'uso dei programmi, né soltanto imparare a scriverli. Secondo la definizione dell'ACM è lo studio sistematico degli *algoritmi* che descrivono e trasformano l'informazione: la loro teoria, analisi, progetto, efficienza, realizzazione e applicazione.

Il corso vuole allenare il **pensiero computazionale**: rendere un problema descrivibile, risolvibile e verificabile da un esecutore. Ha quattro ingredienti:

1. **astrazione**: tenere ciò che conta, ignorare i dettagli;
2. **scomposizione**: dividere un problema grande in problemi piccoli;
3. **riconoscimento di schemi**: accorgersi delle regolarità;
4. **algoritmo e automazione**: scrivere una procedura eseguibile e controllarne l'effetto.

Un **algoritmo** è una sequenza di passi che, eseguita da un esecutore adatto, porta dai dati ai risultati attesi. Ne usiamo tutti i giorni: istruzioni di montaggio, la lavastoviglie, il bancomat. Il nome viene dal matematico persiano al-Khwārizmī (circa 780–850), le cui opere portarono in Europa la numerazione posizionale con lo zero.

Due algoritmi antichissimi. Il papiro di Rhind (circa 1650 a.C.) moltiplica con soli raddoppi, dimezzamenti e somme:

$$A = 1 \Rightarrow A \cdot B = B, \qquad A \text{ pari} \Rightarrow A \cdot B = \frac{A}{2} \cdot (2B), \qquad A \text{ dispari} \Rightarrow A \cdot B = (A-1) \cdot B + B.$$

Euclide calcola il massimo comun divisore senza elencare i divisori, con sottrazioni:

$$\mathrm{MCD}(x, x) = x.$$

$$\mathrm{MCD}(x, y) = \mathrm{MCD}(x - y, y) \text{ se } x > y > 0.$$

$$\mathrm{MCD}(x, y) = \mathrm{MCD}(x, y - x) \text{ se } y > x > 0.$$

Esercizio guidato · Due algoritmi a mano
Calcola $13 \cdot 11$ con il metodo egizio e MCD$(12, 18)$ con le sottrazioni.

Idea. Ogni regola trasforma il problema in uno più piccolo dello stesso tipo, finché si arriva al caso che si sa risolvere subito.

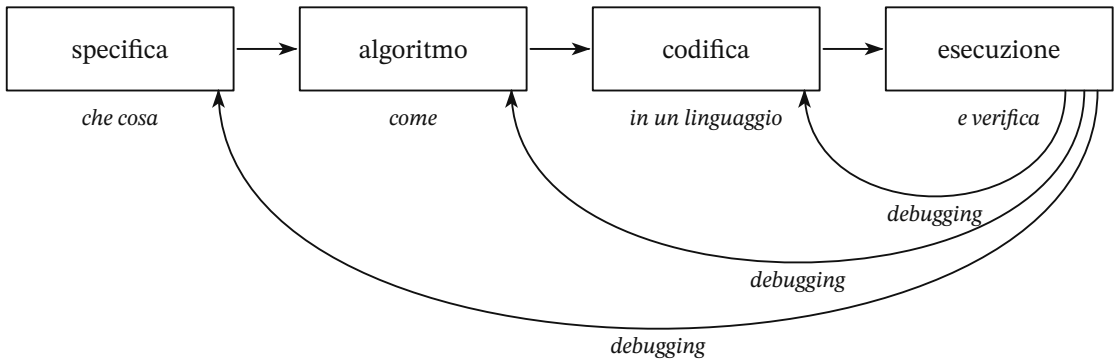
Conto. $13 \cdot 11 = 12 \cdot 11 + 11$; $12 \cdot 11 = 6 \cdot 22 = 3 \cdot 44$; $3 \cdot 44 = 2 \cdot 44 + 44 = 1 \cdot 88 + 44$. Raccogliendo: $13 \cdot 11 = 88 + 44 + 11 = 143$.

Per Euclide: $(12, 18) \to (12, 6) \to (6, 6)$, quindi il MCD è 6.

Controllo. $13 \cdot 11 = 143$ e $12 = 2 \cdot 6$, $18 = 3 \cdot 6$ con $2, 3$ senza fattori comuni.

1.2 Le quattro fasi della programmazione

Programmare significa passare da un problema a un programma che lo risolve. A un primo livello ci sono quattro fasi.



Le quattro fasi. L'esecuzione può rivelare errori nati in qualunque fase precedente: la messa a punto (*debugging*) torna indietro fin dove serve.

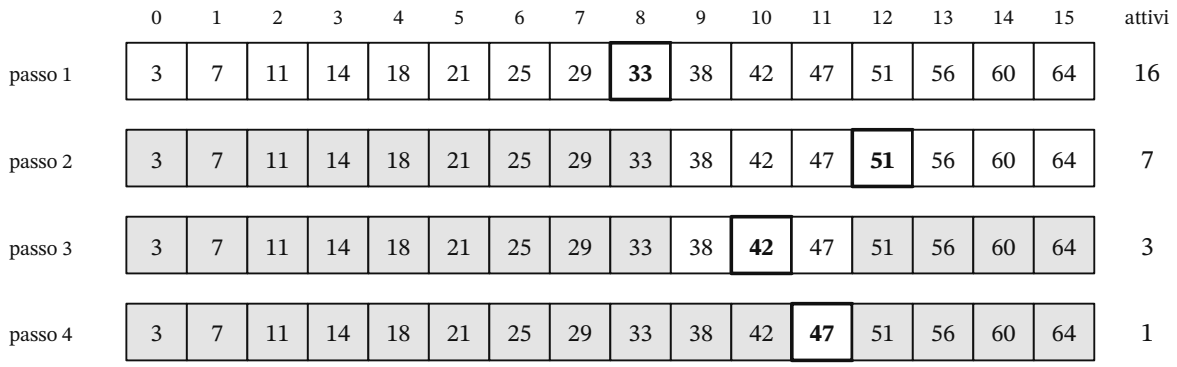
Specifica. Capire e definire il problema: lo **stato iniziale** (i dati, *input*) e lo **stato finale** atteso (i risultati, *output*). Esempi: «dati due numeri, trova il maggiore»; «dati un elenco telefonico e un nome, trova il numero»; «dato un programma e un input, decidi se il programma termina» (il **problema dell'arresto**). Gli esempi mostrano limiti diversi: una specifica non dice *come* risolvere il problema; può essere ambigua (due persone con lo stesso nome); per alcuni problemi non si conosce un metodo (trovare un pari maggiore di 2 che non sia somma di due primi, la congettura di Goldbach); per l'arresto è dimostrato che un metodo generale *non può esistere*. Nel corso ci occupiamo di problemi **decidibili**, per cui un metodo esiste.

Algoritmo. Un procedimento risolutivo è un algoritmo se ha tre requisiti: **finitezza** (un numero finito di passi elementari), **non ambiguità** (ogni passo si interpreta in un solo modo: «sale quanto basta» non va bene per un esecutore automatico) ed **eseguibilità** (ogni passo si completa in tempo finito con le risorse disponibili).

Codifica. Si sceglie come rappresentare gli oggetti e si scrive l'algoritmo in un linguaggio che l'esecutore conosce. Per un calcolatore il linguaggio è un linguaggio di programmazione, e il risultato è un programma con dati in ingresso, dati ausiliari (usati durante il calcolo) e dati in uscita.

Esecuzione. Si esegue il programma. Se il risultato non è quello voluto, la colpa può stare in qualunque fase: una specifica sbagliata, un algoritmo che dimentica un caso, una traduzione scorretta. Gli strumenti che aiutano a trovare gli errori (*bug*) si chiamano **debugger**.

Algoritmi diversi possono risolvere lo stesso problema con efficienza molto diversa. Per cercare un nome in un elenco ordinato, la **ricerca binaria** apre l'elenco a metà, confronta, e continua solo nella metà giusta: a ogni passo scarta metà dei candidati.



Ricerca binaria di 47 fra sedici numeri ordinati. Il riquadro spesso è l'elemento confrontato; il grigio è la parte già scartata. Bastano quattro confronti invece di dodici.

1.3 L'esecutore conta

Specifica: «dati due interi positivi A e B, ottenere $A \cdot B$». Se l'esecutore è una persona con la calcolatrice, l'algoritmo è banale: digita A, il tasto `*`, B, il tasto `=`.

Se invece l'esecutore sa solo sommare, sottrarre e confrontare, serve un altro algoritmo, basato su $A \cdot B = A + A + \cdots + A$ (B volte):

1. scrivi A e B; scrivi 0 in un riquadro ausiliario C;
2. finché $B > 0$: somma A a C e togli 1 a B;
3. il risultato è in C.

Qui A è il valore da sommare, B conta le somme che mancano e C accumula il risultato. Per un bambino che sa fare i conti su un quaderno, i «riquadri» sono tre caselle disegnate sul foglio, in cui si cancella e si riscrive.

> **ATTENZIONE · L'algoritmo dipende dall'esecutore**
> Lo stesso problema richiede algoritmi diversi per esecutori diversi. Un algoritmo è caratterizzato dal problema *e* dalle operazioni elementari che l'esecutore sa eseguire.

1.4 Stati

In ogni momento dell'esecuzione le informazioni rilevanti formano uno **stato**: il contenuto dei riquadri del quaderno, il contenuto della memoria di un calcolatore (modello di von Neumann), il piano a cui si trova un ascensore. L'azione da fare dipende dallo stato: l'ascensore al piano 2 chiamato al 3 sale, al piano 4 scende.

In astratto, uno stato è un insieme di **associazioni** tra nomi simbolici e valori, scritte $X \mapsto \text{val}$:

$$\{\text{nome} \mapsto \text{Antonio}, \text{cognome} \mapsto \text{Rossi}, \text{età} \mapsto 25\}, \qquad \{A \mapsto 25, B \mapsto 350\}.$$

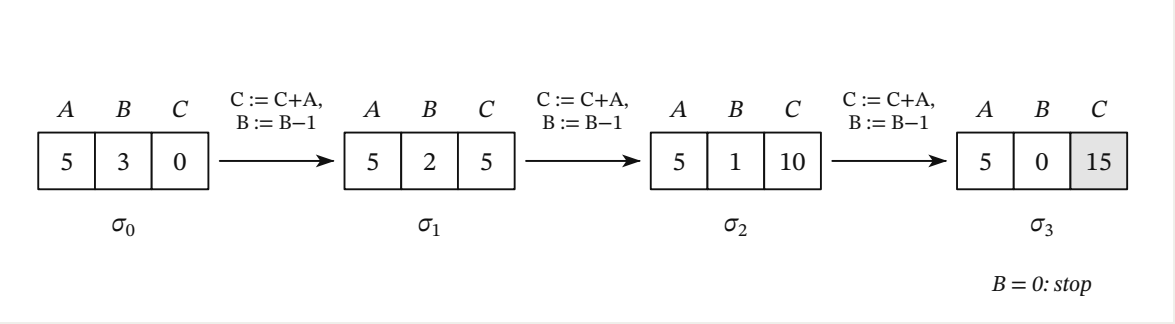
C'è un solo vincolo: **ogni nome ha al più un valore**. $\{\text{nome} \mapsto \text{Antonio}, \text{nome} \mapsto \text{Paolo}\}$ non è uno stato: quale nome useremmo? Un nome può invece mancare del tutto.

Un **algoritmo** è allora una sequenza di passi elementari che modificano lo stato fino a quello desiderato, e un'esecuzione è una successione di stati $S_0 \to S_1 \to \cdots \to S_f$. Nel paradigma imperativo deve esserci almeno un'azione che cambia lo stato: in C sarà l'**assegnamento**.

Esercizio guidato · Eseguire $5 \cdot 3$ per somme
Parti dallo stato $\{A \mapsto 5, B \mapsto 3, C \mapsto 0\}$ ed esegui l'algoritmo.

Idea. A ogni giro si controlla $B > 0$; se è vero, C cresce di A e B cala di 1.

Conto. Gli stati sono quelli della figura.



L'esecuzione come successione di stati: tre giri, poi $B = 0$ e ci si ferma. Il risultato è in C.

Controllo. $C = 15 = 5 \cdot 3$. In aula il primo conto a mano aveva dato 18: era un errore di *esecuzione* (un passo sbagliato), non dell'*algoritmo*. Sono due correttezze diverse.

La specifica del prodotto, scritta con gli stati, è

$$\{\text{fattore1} \mapsto a, \text{fattore2} \mapsto b\} \quad \longrightarrow \quad \{\text{prodotto} \mapsto a \cdot b\}.$$

Qui a e b non sono numeri precisi: sono **variabili di specifica**, nomi per i valori iniziali. La specifica non dice nulla sui valori finali di fattore1, fattore2 o di altri nomi: interessa solo prodotto.

Officina della dimostrazione · 1. Leggere una specifica
Prima di scrivere una riga di algoritmo, rispondi a quattro domande.

- *Che cosa so all'inizio?* Lo stato iniziale: quali nomi, quali valori, quali condizioni («interi positivi»).
- *Che cosa devo ottenere?* Lo stato finale, scritto in funzione dei valori iniziali ($a \cdot b$, non «il prodotto» in generale).
- *Che cosa non mi è chiesto?* I nomi su cui la specifica tace possono finire con qualunque valore.
- *Chi esegue?* Le operazioni elementari disponibili decidono l'algoritmo.

Questo schema tornerà nel capitolo 6, dove la specifica diventa una coppia di formule, e nel capitolo 7, dove diventa una tripla di Hoare.

1.5 Che cosa si può calcolare

Solo dagli anni Trenta del Novecento si è cercato di dire con precisione che cos'è una «procedura effettiva». Alan Turing propose una macchina astratta che manipola simboli secondo un numero finito di regole; mostrò anche una **macchina universale** che riceve la descrizione di un'altra macchina e il suo input e ne simula il calcolo: il modello teorico del computer a programma memorizzato. Alonzo Church propose il λ-calcolo, basato sulle funzioni: un formalismo diversissimo, che però calcola esattamente le stesse funzioni.

La **tesi di Church-Turing** afferma che le macchine di Turing risolvono tutti i problemi effettivamente risolubili. È una *tesi*, accettata da tutti, non un teorema: collega un modello formale a un'idea intuitiva, che non si può dimostrare. Gli altri modelli (sistemi di Post, funzioni ricorsive di Kleene, calcolo quantistico, DNA computing) sono considerati equivalenti quanto a ciò che si può calcolare. Il **modello di von Neumann** (1945), base dei calcolatori attuali, ha la stessa potenza; la sua generalità viene dal programma memorizzato.

I linguaggi di programmazione si dividono per **paradigma**: imperativo (comandi che modificano lo stato: Assembly, FORTRAN, C, Pascal), funzionale (ML, Haskell, Scheme), a oggetti (Java, Smalltalk), logico (Prolog), concorrente (CCS, CSP). Il corso adotta il paradigma imperativo e, più avanti, il linguaggio C.

VERIFICA 1

a) «Aggiungi zucchero finché è dolce»: quale requisito di un algoritmo viola?

b) $\{x \mapsto 3, y \mapsto 3\}$ e $\{x \mapsto 3, x \mapsto 4\}$: quale dei due è uno stato?

c) Con quanti confronti al massimo la ricerca binaria trova un nome fra 1000?

Capitolo 2

Proposizioni: forma e significato

In breve
Per ragionare sui programmi servono frasi senza ambiguità. Il **calcolo proposizionale** è il linguaggio logico più semplice: costruisce frasi composte con i connettivi e stabilisce quando sono vere. La domanda del capitolo: quando una conclusione *segue davvero* dalle premesse?

2.1 Ragionamenti validi

«Tutti gli uomini sono mortali; Socrate è un uomo; quindi Socrate è mortale.» Il ragionamento è **valido**: *se* le premesse sono vere, la conclusione non può essere falsa. Si dice che la conclusione è **conseguenza logica** delle premesse. Non serve sapere se le premesse siano vere nel mondo: conta la forma.

«Tutti gli uomini sono mortali; Socrate è mortale; quindi Socrate è un uomo.» Questo *non* è valido: Socrate potrebbe essere un cane, mortale anche lui. Le premesse vere non bastano; la conclusione deve *seguire*.

La logica nasce per formalizzare il ragionamento (Aristotele, poi la logica moderna di fine Ottocento). In informatica serve a scrivere requisiti, studiare e verificare programmi, rappresentare conoscenza; in Prolog si programma scrivendo premesse e regole. Nel corso la useremo per la **logica di Hoare** (capitoli 7–9).

Una **proposizione** è una frase dichiarativa che è vera o falsa: «Roma è la capitale d'Italia», «La Francia è in Asia» (falsa, ma è una proposizione), $1+1=2$. Non sono proposizioni una domanda, un comando, o $x+1=2$ finché x non è fissato. Lavoriamo nella **logica classica**: ogni proposizione è vera o falsa (*terzo escluso*) e mai entrambe (*non contraddizione*).

Una proposizione è **atomica** se non la scomponiamo; le altre si costruiscono con i **connettivi**, come le espressioni con le operazioni.

2.2 I connettivi

Indichiamo con Bool$=\{\mathsf{t},\mathsf{f}\}$ i valori di verità. Il significato dei connettivi è dato dalla loro tabella.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \equiv Q$
t	t	f	t	t	t	t
t	f	f	f	t	f	f
f	t	t	f	t	t	f
f	f	t	f	f	t	t

$\neg P$ (non P) inverte il valore. $P \wedge Q$ (P e Q) è vera solo se entrambe lo sono. $P \vee Q$ (P o Q) è vera se almeno una lo è: è la disgiunzione *inclusiva*; quella esclusiva («carne o pesce, non entrambi») non la useremo come connettivo. $P \equiv Q$ (P se e solo se Q) è vera quando i due valori coincidono.

IMMAGINE MENTALE · L'implicazione è una promessa
«Se piove, porto l'ombrello.» L'unico modo di *rompere* la promessa è che piova e io non abbia l'ombrello. Se non piove, la promessa è mantenuta qualunque cosa io faccia. Per questo $P \Rightarrow Q$ è falsa solo nella riga $P = \mathsf{t}$, $Q = \mathsf{f}$, e $(P \Rightarrow Q) \equiv \neg(P \wedge \neg Q)$.

2.3 Sintassi: quali stringhe sono formule

Fissato un alfabeto $\mathcal{P} = \{p, q, r, \dots\}$ di simboli proposizionali, le **formule** sono tutte e sole quelle costruite così: T e F sono formule (la formula sempre vera e quella sempre falsa: non sono simboli dell'alfabeto); ogni $p \in \mathcal{P}$ è una formula; se P e Q sono formule, lo sono $\neg P$, $(P \wedge Q)$, $(P \vee Q)$, $(P \Rightarrow Q)$, $(P \equiv Q)$. In informatica si scrive spesso come **grammatica libera dal contesto**:

```
Prop ::= (Prop ≡ Prop) | (Prop ∧ Prop) | (Prop ∨ Prop)
       | (Prop ⇒ Prop) | ¬Prop | T | F | p | q | ...
```

Stringhe come $p \wedge \vee$ o $()$ non sono formule. Le parentesi contano: $(P \Rightarrow \neg P) \Rightarrow Q$ e $P \Rightarrow (\neg P \Rightarrow Q)$ sono formule diverse, e con $P = Q = \mathsf{f}$ la prima è falsa, la seconda vera.

Per toglierne qualcuna si eliminano le parentesi esterne e si usano le **precedenze**, dalla più debole alla più forte: $\equiv$ (0), $\Rightarrow$ (1), $\wedge$ e $\vee$ (2), $\neg$ (3). Così $P \wedge Q \Rightarrow R \equiv R \vee P \Rightarrow S$ si legge $((P \wedge Q) \Rightarrow R) \equiv ((R \vee P) \Rightarrow S)$, e $P \Rightarrow Q \equiv \neg P \vee Q$ si legge $(P \Rightarrow Q) \equiv (\neg P \vee Q)$.

ATTENZIONE · Fra $\wedge$ e $\vee$ non c'è precedenza
$P \wedge Q \vee R$ è *sbagliata*: $(P \wedge Q) \vee R$ e $P \wedge (Q \vee R)$ non sono equivalenti (con $P = \mathsf{f}$, $R = \mathsf{t}$ la prima è vera e la seconda falsa). Anche $P \Rightarrow Q \Rightarrow R$ è sbagliata se non si dice come associare. $P \wedge Q \wedge R$ è invece accettata, perché $\wedge$ è associativo.

2.4 Tradurre le frasi

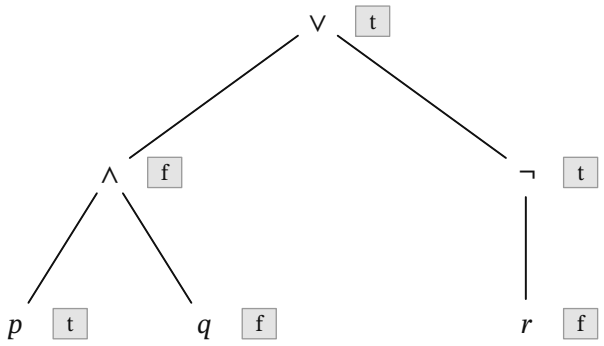
Con p: piove, n: ci sono nuvole, v: c'è vento, r: fa freddo, c: ci si copre, ne: nevica.

Frase	Formula
Fa freddo, ma non piove	$r \wedge \neg p$
Se ci sono nuvole e non c'è vento, allora piove	$(n \wedge \neg v) \Rightarrow p$
Nevica, ma non fa freddo se ci si copre	$\mathrm{ne} \wedge (c \Rightarrow \neg r)$
Se ci si copre, allora fa freddo o nevica	$c \Rightarrow (r \vee \mathrm{ne})$
Piove solo se ci sono nuvole e non c'è vento	$p \Rightarrow (n \wedge \neg v)$

«Ma» si traduce con $\wedge$. E soprattutto: «P *solo se* Q» è $P \Rightarrow Q$, perché Q è una **condizione necessaria** per P. Anche «condizione necessaria perché Antonio vada al cinema è che ci vada Bruno» diventa $a \Rightarrow b$.

2.5 Interpretazioni e valore di una formula

Il valore di una formula dipende dai valori dei simboli, come il valore di $x + y$ dipende da x e y. Un'**interpretazione** è una funzione $I : \mathcal{P} \to \text{Bool}$, per esempio $I = \{p \mapsto \mathsf{t}, q \mapsto \mathsf{f}, r \mapsto \mathsf{f}\}$. Il valore $[\![\varphi]\!]_I$ si calcola *ricorsivamente*: $[\![T]\!]_I = \mathsf{t}$, $[\![F]\!]_I = \mathsf{f}$, $[\![p]\!]_I = I(p)$, e per ogni connettivo si applica la sua tabella ai valori delle parti, per esempio $[\![P \wedge Q]\!]_I = [\![P]\!]_I \wedge [\![Q]\!]_I$. A destra il simbolo indica la funzione booleana: lo stesso segno serve per la sintassi e per il significato.



La formula $(p \wedge q) \vee \neg r$ come albero. Con $I = \{p \mapsto \mathsf{t}, q \mapsto \mathsf{f}, r \mapsto \mathsf{f}\}$ si valuta dal basso: $p \wedge q = \mathsf{f}$, $\neg r = \mathsf{t}$, quindi la radice (il *connettivo principale*) vale t.

Una **tavola di verità** elenca il valore di una formula in tutte le interpretazioni dei suoi simboli: con n simboli sono 2^n righe.

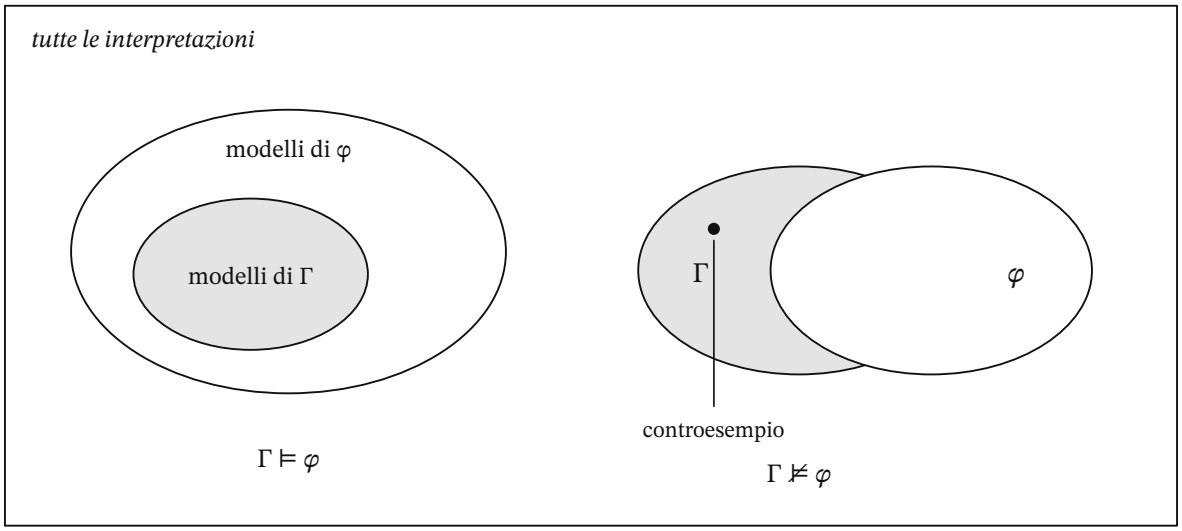
p	q	r	$p \wedge q$	$\neg r$	$(p \wedge q) \vee \neg r$
t	t	t	t	f	t
t	t	f	t	t	t
t	f	t	f	f	f
t	f	f	f	t	t
f	t	t	f	f	f
f	t	f	f	t	t
f	f	t	f	f	f
f	f	f	f	t	t

2.6 Tautologie e conseguenza logica

Un'interpretazione I è un **modello** di φ, e si scrive $I \vDash \varphi$, se $[\![\varphi]\!]_I = \mathsf{t}$; è modello di un insieme Γ se lo è di ogni sua formula. Una formula è

- **soddisfacibile** se ha almeno un modello;
- una **tautologia** ($\vDash \varphi$) se ogni interpretazione è un modello: per esempio $p \vee \neg p$;
- una **contraddizione** (insoddisfacibile) se non ha modelli: per esempio $p \wedge \neg p$.

φ è una tautologia se e solo se $\neg\varphi$ è una contraddizione. Infine φ è **conseguenza logica** di Γ, $\Gamma \vDash \varphi$, se ogni modello di Γ è anche modello di φ. Con premesse finite basta controllare che $(\varphi_1 \wedge \cdots \wedge \varphi_n) \Rightarrow \varphi$ sia una tautologia.



A sinistra $\Gamma \vDash \varphi$: i modelli delle premesse stanno tutti dentro i modelli della conclusione. A destra no: un'interpretazione rende vere le premesse e falsa la conclusione, ed è un *controesempio*.

ESERCIZIO GUIDATO · Gli amici al cinema

Se Corrado va al cinema ci va anche Antonio ($c \Rightarrow a$); condizione necessaria perché Antonio vada è che vada Bruno ($a \Rightarrow b$). Quali conclusioni seguono? (1) $c \Rightarrow b$; (2) $\neg a \wedge \neg b \wedge \neg c$; (3) $b \Rightarrow c$; (4) $\neg c \Rightarrow \neg b$.

Idea. Per escludere una conclusione basta *un* controesempio: un'interpretazione che renda vere le due premesse e falsa la conclusione. Per accettarla serve che non esista.

Conto. (1) Se $c = \mathsf{t}$, dalla prima premessa $a = \mathsf{t}$, dalla seconda $b = \mathsf{t}$: la conclusione non può essere falsa. Segue (è la *transitività*).

(2) Tutti al cinema ($a = b = c = \mathsf{t}$) rende vere le premesse e falsa la conclusione. Non segue.

(3) $b = \mathsf{t}$, $c = \mathsf{f}$ (e a qualsiasi): $c \Rightarrow a$ è vera perché c è falsa, $a \Rightarrow b$ è vera perché b è vera, ma $b \Rightarrow c$ è falsa. Non segue.

(4) $\neg c \Rightarrow \neg b$ ha la stessa tabella di $b \Rightarrow c$: lo stesso controesempio la esclude.

Controllo. Solo la (1) segue. Nella (4) si riconosce la *controposizione* di (3), che vedremo nel capitolo 3.

Lo stesso metodo vale per le frasi di tutti i giorni. «Se piove non esco; ieri sono uscito; quindi non pioveva» è $((P \Rightarrow \neg E) \wedge E) \Rightarrow \neg P$: una tautologia, ragionamento corretto. «Se piove non esco; ieri non pioveva; quindi sono uscito» è $((P \Rightarrow \neg E) \wedge \neg P) \Rightarrow E$: con $P = E = \mathsf{f}$ le premesse sono vere e la conclusione falsa. Non è corretto.

Officina della dimostrazione · 2. Basta un caso o servono tutti?
Prima di metterti a calcolare, chiediti se la frase è un «esiste» o un «per ogni» nascosto.

- *Soddisfacibile?* È un «esiste»: basta trovare un'interpretazione che la renda vera.
- *Non è una tautologia? Non è conseguenza logica?* Basta un controesempio. Conviene partire dalla fine: rendi falsa la conclusione e prova a rendere vere tutte le premesse.
- *È una tautologia? È conseguenza logica?* È un «per ogni»: serve tutta la tavola (2^n righe) o una dimostrazione (capitolo 3).

Se la ricerca del controesempio fallisce per forza (come per $((a \Rightarrow b) \wedge a) \Rightarrow b$: con $a = \mathsf{t}$, $b = \mathsf{f}$ la premessa $a \Rightarrow b$ diventa falsa), quel fallimento è già un indizio che la formula è una tautologia.

Verifica 2

a) Traduci «esco solo se non piove».

b) $p \Rightarrow q$ è soddisfacibile? È una tautologia?

c) Trova un controesempio per $((a \Rightarrow b) \wedge \neg a) \Rightarrow \neg b$.

Capitolo 3

Dimostrare con le leggi

In breve
Le tavole di verità funzionano, ma con n simboli hanno 2^n righe, e nella logica del primo ordine i casi diventano infiniti. Impariamo allora a **dimostrare**: trasformare una formula passo dopo passo con leggi già note, giustificando ogni passo. È esattamente il lavoro che faremo poi sui programmi.

3.1 Dimostrare invece di provare tutto

Una **regola di inferenza** permette di derivare un asserto da altri. Due esempi: dal *modus ponens*, da $Q \Rightarrow R$ e Q si deduce R; dall'*istanziazione*, da «per ogni x vale $P(x)$» si deduce $P(a)$. Così si dimostra che Socrate è mortale: istanziando «per ogni x, se x è un uomo allora è mortale» su Socrate, e applicando il modus ponens con «Socrate è un uomo». Lo stesso schema dimostra che 8 non è primo: cambia il significato dei simboli, non la forma.

Una dimostrazione è quindi *sintattica*: manipola simboli con regole meccaniche, come un algoritmo. La conseguenza logica è invece *semantica*: parla di verità nelle interpretazioni. Le due cose si incontrano se le regole sono

- **corrette**: tutto ciò che si dimostra è conseguenza logica (non si deriva mai il falso da premesse vere);
- **complete**: ogni conseguenza logica si può dimostrare.

Un esempio familiare dall'algebra: ogni passo cita la legge usata.

$$
\begin{aligned}
& (a+b)(a-b) \\
=\ & a(a-b)+b(a-b) && \{\text{distributività}\} \\
=\ & a^2 - a\,b + b\,a - b^2 && \{\text{distributività, associatività}\} \\
=\ & a^2 - b^2 && \{\text{commutatività, opposto, elemento neutro}\}
\end{aligned}
$$

Ogni passo usa anche il **principio di sostituzione**: se $A = B$, sostituire A con B dentro un'espressione C non ne cambia il valore. Nel calcolo proposizionale vale l'analogo per l'equivalenza:

$$\frac{P \equiv Q}{R \equiv R[Q/P]}$$

dove $R[Q/P]$ è R con un'occorrenza di P sostituita da Q. Una **legge** è una tautologia che usiamo come giustificazione.

3.2 Le leggi

Ogni legge si verifica con una tavola di verità (basta usare simboli p, q, r: la tavola vale per ogni

formula al loro posto) o con leggi già dimostrate. Non vanno imparate a memoria: servono a riconoscere i passi.

Nome	Legge
Equivalenza	$P \equiv P$ $(P \equiv Q) \equiv (Q \equiv P)$ $(P \equiv T) \equiv P$ (unità), associatività, transitività
Comm., ass., idem.	$P \vee Q \equiv Q \vee P$ $P \vee (Q \vee R) \equiv (P \vee Q) \vee R$ $P \vee P \equiv P$ (e lo stesso per $\wedge$)
Unità e zero	$P \wedge T \equiv P$ $P \vee F \equiv P$ $P \wedge F \equiv F$ $P \vee T \equiv T$
Distributività	$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$ $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$
Negazione	$\neg\neg P \equiv P$ $P \vee \neg P \equiv T$ (terzo escluso) $P \wedge \neg P \equiv F$ $\neg T \equiv F$
De Morgan	$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
Eliminazione di $\Rightarrow$	$P \Rightarrow Q \equiv \neg P \vee Q$ $P \Rightarrow Q \equiv \neg(P \wedge \neg Q)$ $\neg(P \Rightarrow Q) \equiv P \wedge \neg Q$
Eliminazione di $\equiv$	$(P \equiv Q) \equiv (P \Rightarrow Q) \wedge (Q \Rightarrow P)$ $(P \equiv Q) \equiv (P \wedge Q) \vee (\neg P \wedge \neg Q)$
Complemento	$P \vee (\neg P \wedge Q) \equiv P \vee Q$ $P \wedge (\neg P \vee Q) \equiv P \wedge Q$
Assorbimento	$P \wedge (P \vee Q) \equiv P$ $P \vee (P \wedge Q) \equiv P$
Implicative	$(P \Rightarrow Q) \wedge P \Rightarrow Q$ (modus ponens) $(P \Rightarrow Q) \wedge (Q \Rightarrow R) \Rightarrow (P \Rightarrow R)$ $P \wedge Q \Rightarrow P$ $P \Rightarrow P \vee Q$
Scambio	$(P \wedge Q \Rightarrow R) \equiv (P \wedge \neg R \Rightarrow \neg Q)$

3.3 Catene di equivalenze

Per dimostrare $P_1 \equiv P_n$ si scrive una catena $P_1 \equiv P_2 \equiv \cdots \equiv P_n$, con ogni passo giustificato tra graffe. Ecco la legge di complemento:

$$
\begin{array}{ll}
 & P \vee (\neg P \wedge Q) \\
\equiv & (P \vee \neg P) \wedge (P \vee Q) \qquad \{\text{distributività}\} \\
\equiv & T \wedge (P \vee Q) \qquad \{\text{terzo escluso}\} \\
\equiv & P \vee Q \qquad \{\text{unità}\}
\end{array}
$$

e l'assorbimento, con un trucco: aggiungere un F che non cambia nulla.

$$\begin{aligned} & P \wedge (P \vee Q) \\ \equiv\ & (P \vee F) \wedge (P \vee Q) && \{\text{unità}\} \\ \equiv\ & P \vee (F \wedge Q) && \{\text{distributività}\} \\ \equiv\ & P \vee F && \{\text{zero}\} \\ \equiv\ & P && \{\text{unità}\} \end{aligned}$$

La tecnica non sceglie per noi la legge o il lato da cui partire. Tre strategie aiutano: trasformare P fino a Q; ridurre P e Q a una stessa formula R; se si vuole una tautologia, ridurla a T. Passi ovvi di associatività, commutatività e idempotenza si possono omettere.

ATTENZIONE · Si manipolano simboli, non valori
Nella legge di unità compare la *costante* F, non un simbolo qualsiasi f. E quando si elimina $\Rightarrow$ si rispetta la struttura: $P \Rightarrow (Q \wedge R) \equiv \neg P \vee (Q \wedge R)$, non $(\neg P \vee Q) \wedge R$.

ESERCIZIO GUIDATO · Il modus ponens è una tautologia
Dimostra $((P \Rightarrow Q) \wedge P) \Rightarrow Q \equiv T$.

Idea. Togliere le implicazioni, poi semplificare finché resta un terzo escluso.

Conto.

$$\begin{aligned} & ((P \Rightarrow Q) \wedge P) \Rightarrow Q \\ \equiv\ & \neg(P \wedge (\neg P \vee Q)) \vee Q && \{\text{elim. } \Rightarrow \text{ due volte, commutatività}\} \\ \equiv\ & \neg(P \wedge Q) \vee Q && \{\text{complemento}\} \\ \equiv\ & \neg P \vee \neg Q \vee Q && \{\text{De Morgan}\} \\ \equiv\ & \neg P \vee T && \{\text{terzo escluso}\} \\ \equiv\ & T && \{\text{zero}\} \end{aligned}$$

Controllo. Cercando un controesempio ($Q = \mathsf{f}$, premessa vera) si è costretti a $P = \mathsf{t}$, ma allora $P \Rightarrow Q$ è falsa: nessun controesempio, come previsto.

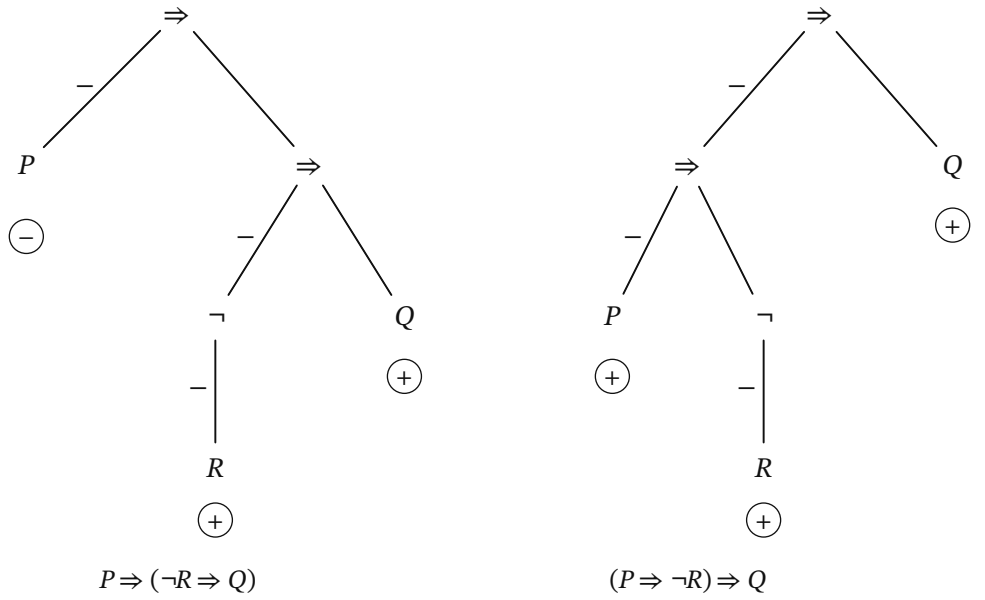
3.4 Implicazioni e polarità

Nella verifica dei programmi servono soprattutto implicazioni. Si possono costruire catene miste, con passi $\equiv$ e passi $\Rightarrow$ giustificati da leggi implicative. Per esempio il *tollendo tollens*:

$$\begin{aligned} & (P \Rightarrow Q) \wedge \neg Q \\ \equiv\ & (\neg P \vee Q) \wedge \neg Q && \{\text{elim. } \Rightarrow\} \\ \equiv\ & \neg P \wedge \neg Q && \{\text{complemento}\} \\ \Rightarrow\ & \neg P && \{\text{semplificazione}\} \end{aligned}$$

Qui serve cautela. Da $P \Rightarrow Q$ *non* segue in generale $R \Rightarrow R[Q \,/\, P]$. La legge $P \wedge Q \Rightarrow P$ non permette di passare da studia∧frequenta⇒supera a studia⇒supera: studiare da solo non basta. È come con le disuguaglianze: sotto un segno meno il verso si inverte. In logica fanno da «segno meno» la negazione e l'antecedente di un'implicazione.

Un'occorrenza è **positiva** o **negativa** secondo quanti passaggi negativi la separano dalla radice della formula: pari positiva, dispari negativa. Prima si elimina ogni $\equiv$: l'equivalenza non è monotona nei suoi argomenti. I passaggi negativi sono l'argomento di $\neg$ e il lato sinistro di $\Rightarrow$; tutto il resto ($\wedge$, $\vee$, lato destro di $\Rightarrow$) non cambia segno.



Due formule come alberi; i rami segnati − invertono il segno. In $P \Rightarrow (\neg R \Rightarrow Q)$ l'occorrenza di P è negativa, R e Q positive. In $(P \Rightarrow \neg R) \Rightarrow Q$ sono tutte positive: P e R attraversano due rami negativi.

Le equivalenze lo confermano: $P \Rightarrow (\neg R \Rightarrow Q) \equiv \neg P \vee R \vee Q$ e $(P \Rightarrow \neg R) \Rightarrow Q \equiv (P \wedge R) \vee Q$. Il **principio di sostituzione per l'implicazione** dice: se vale $P \Rightarrow Q$, allora $R \Rightarrow R[Q/P]$ quando l'occorrenza sostituita è positiva, e $R[Q/P] \Rightarrow R$ quando è negativa.

Due esempi. Da $Q \wedge R \Rightarrow Q$, poiché $Q \wedge R$ è positiva nel conseguente, $(P \Rightarrow Q \wedge R) \Rightarrow (P \Rightarrow Q)$. Da $P \Rightarrow P \vee Q$, poiché P è negativa nell'antecedente di $P \Rightarrow R$, il verso si inverte: $(P \vee Q \Rightarrow R) \Rightarrow (P \Rightarrow R)$.

3.5 Le tecniche di dimostrazione sono tautologie

Molte tecniche che si usano in matematica sono leggi del calcolo proposizionale.

- **Per assurdo**: $P \equiv (\neg P \Rightarrow F)$, e per un'implicazione $(Q \Rightarrow R) \equiv (Q \wedge \neg R \Rightarrow F)$: si assumono l'ipotesi e la negazione della tesi e si arriva a una contraddizione.
- **Per controposizione**: $(P \Rightarrow Q) \equiv (\neg Q \Rightarrow \neg P)$.
- **Per casi**: $(P \Rightarrow Q) \wedge (\neg P \Rightarrow Q) \equiv Q$; si sceglie un P comodo e si dimostra Q in entrambi i casi.
- **Assumere la premessa**: per dimostrare $P \Rightarrow Q$ si può supporre P e arrivare a Q, citando P tra le giustificazioni. Per esempio, sotto l'ipotesi P, $P \wedge Q \equiv T \wedge Q \equiv Q$; quindi $P \Rightarrow (P \wedge Q \equiv Q)$.

Per esempio, $(P \vee Q \Rightarrow R) \Rightarrow (P \Rightarrow R)$ per assurdo: si nega tutto e si arriva a F.

$$
\begin{array}{ll}
\neg((P \vee Q \Rightarrow R) \Rightarrow (P \Rightarrow R)) & \\
\equiv (\neg(P \vee Q) \vee R) \wedge P \wedge \neg R & \{\text{elim. } \Rightarrow, \text{ negazione di } \Rightarrow\} \\
\Rightarrow \neg(P \vee Q) \wedge P & \{\text{complemento su } R, \text{ semplificazione}\} \\
\equiv \neg P \wedge \neg Q \wedge P & \{\text{De Morgan}\} \\
\equiv F & \{\text{contraddizione}\}
\end{array}
$$

In generale, un passo P conn Q giustificato da G è corretto se $G \Rightarrow (P \text{ conn } Q)$ è una tautologia. Se tutte le giustificazioni sono leggi, la catena dimostra la sua conclusione; se alcune sono ipotesi H, dimostra $H \Rightarrow$ conclusione.

ESERCIZIO GUIDATO · Combinare ipotesi: congiunzione e disgiunzione

Idea. Da $P \Rightarrow Q$ e $R \Rightarrow S$ possiamo trasportare ciascuna componente, sia in una congiunzione sia in una disgiunzione.

Conto. Sotto le due ipotesi, $P \wedge R \Rightarrow Q \wedge R \Rightarrow Q \wedge S$ e $P \vee R \Rightarrow Q \vee R \Rightarrow Q \vee S$. Le occorrenze sostituite sono positive. Quindi $(P \Rightarrow Q) \wedge (R \Rightarrow S) \Rightarrow (P \wedge R \Rightarrow Q \wedge S)$; aggiungendo $P \vee R$ alle premesse si ottiene il sillogismo disgiuntivo, $Q \vee S$.

Un altro esercizio usa $H \equiv (P \Rightarrow (Q \equiv R))$. Sotto H, da $P \wedge R$ si ottiene $(Q \equiv R) \wedge R$, quindi Q, perché l'equivalenza contiene $R \Rightarrow Q$. Abbiamo dimostrato $H \Rightarrow (P \wedge R \Rightarrow Q)$.

Controllo. Le conclusioni dipendono dalle ipotesi: nel testo della dimostrazione bisogna dichiararle. Per spostarle a sinistra usiamo $P \Rightarrow (Q \Rightarrow R) \equiv (P \wedge Q \Rightarrow R)$.

3.6 Forme normali e risoluzione

Una **forma normale congiuntiva** (FNC) è un «e di o», come $(p \vee \neg q) \wedge (q \vee r)$; una **forma normale disgiuntiva** (FND) è un «o di e». Ogni formula si porta in queste forme (eliminando $\Rightarrow$ e $\equiv$, spingendo le negazioni con De Morgan, distribuendo), anche se la formula può crescere molto. Servono per confrontare formule: $\neg((P \Rightarrow Q) \wedge (R \Rightarrow P))$ e $\neg(\neg Q \Rightarrow \neg P) \vee \neg(R \Rightarrow P)$ diventano entrambe $(P \wedge \neg Q) \vee (R \wedge \neg P)$, quindi sono equivalenti.

La **risoluzione** elimina un simbolo presente positivo in una clausola e negato in un'altra:

$$(P \vee Q) \wedge (\neg P \vee R) \Rightarrow Q \vee R.$$

Deriva dalla transitività, perché $(P \vee Q) \wedge (\neg P \vee R) \equiv (\neg Q \Rightarrow P) \wedge (P \Rightarrow R)$. Applicata due volte dimostra $(P \vee Q) \wedge (\neg P \vee R) \wedge (\neg Q \vee R) \Rightarrow R$: si ottiene $(Q \vee R) \wedge (\neg Q \vee R)$, poi $R \wedge R \equiv R$. È il motore della programmazione logica.

Infine, non servono tutti i connettivi: $\{\wedge, \neg\}$ è **funzionalmente completo**, perché $P \vee Q \equiv \neg(\neg P \wedge \neg Q)$ e $P \Rightarrow Q \equiv \neg(P \wedge \neg Q)$. Lo sono anche $\{\vee, \neg\}$, $\{\Rightarrow, \neg\}$ e perfino il solo **NAND**, $P \uparrow Q \equiv \neg(P \wedge Q)$. Teniamo comunque $\Rightarrow$ e $\equiv$ perché rendono leggibili le dimostrazioni.

Officina della dimostrazione · 3. Scrivere una catena

Una dimostrazione è una catena di passi, uno per riga, ciascuno con la sua giustificazione tra graffe.

- Parti dal lato più complicato: è più facile semplificare che complicare.
- Per una tautologia, riducila a T. Per un'implicazione $P \Rightarrow Q$, puoi assumere P e trasformare fino a Q.
- Ogni passo $\Rightarrow$ dentro una formula più grande: controlla la polarità dell'occorrenza che sostituisci.
- Se ti blocchi: elimina $\Rightarrow$ e $\equiv$, spingi le negazioni verso l'interno, cerca un complemento o un terzo escluso.

Le stesse catene, con le stesse graffe, torneranno nei capitoli 7–9 per dimostrare che un programma è corretto.

Verifica 3

a) In $\neg(P \Rightarrow Q) \wedge R$, le occorrenze di P, Q, R sono positive o negative?

b) Dimostra $P \wedge (\neg P \vee Q) \equiv P \wedge Q$ con una catena.

c) Esprimi $\neg P$ usando solo il NAND.

Il linguaggio del primo ordine

In breve
Nel calcolo proposizionale «Alberto va al cinema» è un atomo chiuso: non vediamo né Alberto né il cinema. La **logica del primo ordine** (LPO) apre gli atomi: parla di individui, delle loro proprietà e relazioni, e dice «per ogni» e «esiste». In questo capitolo impariamo a scrivere e a leggere le sue formule.

4.1 Perché il calcolo proposizionale non basta

«Alberto va al cinema con Bruno o va al teatro con Carlo» diventa (AC∧BC) ∨ (AT∧CT): le persone e la relazione «andare a» spariscono. E non si possono scrivere in modo compatto frasi su insiemi infiniti («tutti i pari maggiori di 2 non sono primi»), proprietà che dipendono da una variabile («se x è pari, $x+1$ è dispari») o su molti individui («tutti gli studenti di FPL vanno al cinema»). Nella LPO:

$$\forall x.(\text{pari}(x) \wedge x > 2 \Rightarrow \neg\text{primo}(x)), \qquad \forall x.(\text{pari}(x) \Rightarrow \text{dispari}(x+1)).$$

Una proprietà generale permette di ricavare i casi particolari: dalla seconda segue pari(4) ⇒ dispari(5). Le formule vere in ogni interpretazione si dicono **valide**, come le tautologie; ma ora non esistono tavole finite che decidano la validità: servono dimostrazioni, e non sempre se ne trova una.

4.2 Termini e formule

L'alfabeto ha quattro tipi di simboli, più connettivi, quantificatori ∀, ∃ e punteggiatura.

Simboli	Che cosa indicano
costanti $a, b, c, 2,$ FPL	individui precisi del dominio
variabili x, y, z	individui generici
funzioni $f, g, +, ,$ succ	operatori che da individui danno un individuo; ognuno ha la sua *arietà* (numero di argomenti)
predicati $p, q, =, <,$ pari	proprietà o relazioni; arietà 0 per le lettere proposizionali («piove»)

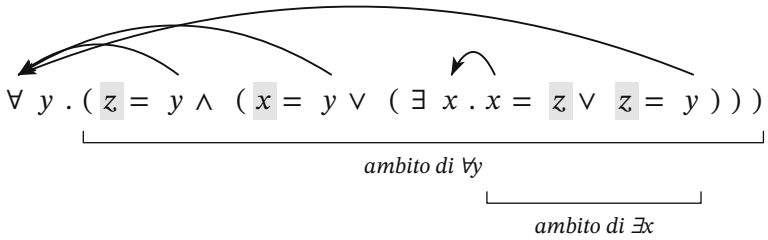
I **termini** denotano individui e si definiscono per induzione: ogni costante e ogni variabile è un termine; se g ha arietà n e $t_1, \ldots, t_n$ sono termini, $g(t_1, \ldots, t_n)$ è un termine. Esempi: a, x, $g(a)$, $h(x, g(a))$, e $x + (1\, z)$, che in notazione prefissa è $+(x, (1, z))$.

Le **formule** denotano valori di verità: T e F; le formule **atomiche** $p(t_1, \ldots, t_n)$; e poi (P), $\neg P$, $P \wedge Q$, $P \vee Q$, $P \Rightarrow Q$, $P \equiv Q$, $(\forall x.P)$, $(\exists x.P)$. Quantificatori uguali di seguito si abbreviano: $\forall x, y, z.P$.

Attenzione · Termini e formule sono cose diverse
$x+1$ è un *termine*: denota un numero. dispari$(x+1)$ è una *formula*: è vera o falsa. Le funzioni trasformano termini in termini; i predicati trasformano termini in formule. Scrivere pari$(x)+1$ non ha senso.

4.3 Variabili libere e legate

In $(\forall x.P)$ la formula P è la **portata** del quantificatore. Un'occorrenza di x è **legata** se sta nella portata di un $\forall x$ o $\exists x$, altrimenti è **libera**. Conta la singola occorrenza: in $(\forall x.p(a,x)) \wedge q(x,y)$ la prima x è legata, la seconda è libera, come y.



Le occorrenze in $\forall y.(z = y \wedge (x = y \vee (\exists x.x = z \vee z = y)))$. Le frecce collegano le variabili legate al loro quantificatore; le occorrenze in grigio sono libere. La formula è aperta, con variabili libere x e z.

Una formula è **chiusa** se non ha variabili libere, **aperta** altrimenti; $P(x)$ ricorda che x è libera in P. Il valore di una formula aperta non è determinato: sui naturali multiplo$(x,3)$ è vera se x «sta per» 6, falsa se sta per 7 (come $x+1=2$ nel capitolo 2). Invece $\forall x.\text{multiplo}(x,3)$ è chiusa, e sui naturali è falsa. Ci occuperemo quasi solo di formule chiuse.

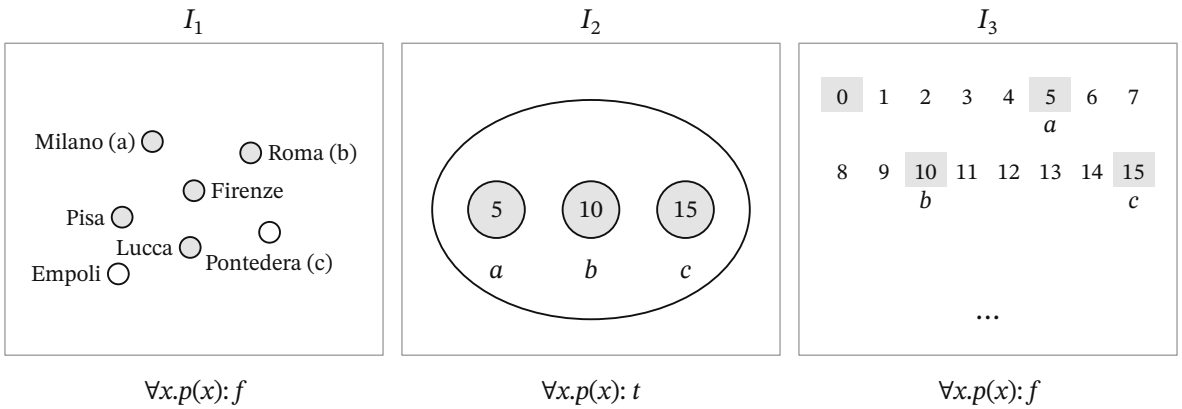
Le variabili legate sono segnaposto e si possono ribattezzare: $(\forall x.P) \equiv (\forall y.P[y/x])$ e $(\exists x.P) \equiv (\exists y.P[y/x])$, purché y non compaia in P. In $P[y/x]$ si sostituiscono solo le occorrenze libere di x in P, non quelle già legate da un quantificatore interno.

> **Attenzione · La cattura**
> La condizione su y è essenziale. In $\forall x.x \le y$ non si può ribattezzare x in y: si otterrebbe $\forall y.y \le y$, dove la y che era libera è stata «catturata» e il significato è cambiato.

4.4 Interpretazioni

Una stessa formula può essere vera o falsa a seconda di come leggiamo i simboli. $\forall x.p(x) \vee q(x)$ è vera sulle persone se p è «maggiorenne» e q è «minorenne»; è falsa sui naturali se p è «primo» e q è «pari» (9 non è né l'uno né l'altro).

Un'**interpretazione** è una coppia $\mathcal{I} = (D, \alpha)$: un **dominio** D (un insieme) e una funzione α che dà a ogni costante un elemento di D, a ogni funzione di arietà n una funzione $D^n \to D$, a ogni predicato di arietà n una funzione $D^n \to \text{Bool}$ (di arietà 0: un valore di verità).



Tre interpretazioni dello stesso alfabeto (costanti a, b, c, un predicato p). In grigio gli elementi che soddisfano p. $\mathcal{I}_2$ e $\mathcal{I}_3$ differiscono solo per il dominio, eppure $\forall x.p(x)$ è vera nella prima e falsa nella seconda.

Formula	$\mathcal{I}_1$ (città, capoluogo)	$\mathcal{I}_2$ ($\{5, 10, 15\}$)	$\mathcal{I}_3$ ($\mathbb{N}$, multiplo di 5)
$p(c)$	f	t	t
$\exists x.p(x)$	t	t	t
$\forall x.p(x)$	f	t	f
$(\exists x.p(x)) \wedge (\exists y.\neg p(y))$	t	f	t

In $\mathcal{I}_1$ le costanti sono Milano, Roma, Pontedera; in $\mathcal{I}_2$ e $\mathcal{I}_3$ sono 5, 10, 15. Il dominio conta anche per gli individui *senza nome*: in $\mathbb{N}$ c'è 7, che nessuna costante nomina ma che basta a rendere falsa $\forall x.p(x)$.

4.5 Formalizzare

Per formalizzare una frase si sceglie un alfabeto e un'interpretazione, e si scrive una formula vera *se e solo se* la frase è vera:

1. individua il dominio D;
2. per ogni individuo nominato, una costante;
3. per ogni operatore che dà un individuo, un simbolo di funzione;
4. per ogni proprietà o relazione, un predicato;
5. scrivi la formula.

«Se un naturale è pari, lo è anche il successore del suo successore»: dominio $\mathbb{N}$, funzione succ con $\alpha(\text{succ})(n) = n + 1$ (un operatore, non un predicato), predicato pari:

$$\forall x.\text{pari}(x) \Rightarrow \text{pari}(\text{succ}(\text{succ}(x))).$$

«Due persone sono parenti se hanno un antenato in comune»: l'antenato non ha nome, quindi diventa una variabile quantificata *dentro* la premessa:

$$\forall x, y.(\exists z.\text{antenato}(z, x) \wedge \text{antenato}(z, y)) \Rightarrow \text{parenti}(x, y).$$

Spesso si quantifica solo su chi soddisfa una condizione P, il **dominio** del quantificatore. Si abbrevia

$$(\forall x{:}\, P.Q) \text{ per } (\forall x.P \Rightarrow Q), \qquad (\exists x{:}\, P.Q) \text{ per } (\exists x.P \wedge Q).$$

ATTENZIONE · Con $\forall$ va $\Rightarrow$, con $\exists$ va $\wedge$
«Qualche studente di FPL va al cinema» è $\exists x.\text{stud}(x) \wedge \text{cin}(x)$. Scrivere $\exists x.\text{stud}(x) \Rightarrow \text{cin}(x)$ è un errore: basta una persona che *non* è studente per rendere vera l'implicazione, e la formula diventa vera quasi sempre. Viceversa $\forall x.\text{stud}(x) \wedge \text{cin}(x)$ direbbe che *tutti* sono studenti.

ESERCIZIO GUIDATO · Tutti tranne uno
Dominio: le persone; costanti A, B, C (Alberto, Bruno, Carlo); predicati segueFPL, vaCinema, $=$. Formalizza: (1) «Alberto non segue FPL ma va al cinema con Bruno o con Carlo»; (2) «tutti gli studenti di FPL vanno al cinema»; (3) «tutti gli studenti di FPL tranne uno vanno al cinema».

Idea. «Ma» è una congiunzione; «tranne uno» dice due cose: ce n'è uno che non ci va, e *tutti gli altri* ci vanno.

Conto. (1) $\neg\text{segueFPL}(A) \wedge \text{vaCinema}(A) \wedge (\text{vaCinema}(B) \vee \text{vaCinema}(C))$.

(2) $\forall x.\text{segueFPL}(x) \Rightarrow \text{vaCinema}(x)$.

(3) $\exists x.(\text{segueFPL}(x) \wedge \neg\text{vaCinema}(x) \wedge \forall y.(\text{segueFPL}(y) \wedge \neg(x = y) \Rightarrow \text{vaCinema}(y)))$.

Controllo. Senza la parte $\forall y$ la (3) direbbe solo «almeno uno non ci va». Senza $\neg(x = y)$ la parte $\forall y$ si applicherebbe anche a x stesso e contraddirebbe $\neg\text{vaCinema}(x)$. Un solo predicato segueFPL è più compatto di $\text{studente}(x) \wedge \text{frequenta}(x, \text{FPL})$, ma non permette di parlare di altri corsi: l'alfabeto si sceglie in base a ciò che si vuole dire.

Officina della dimostrazione · 4. Formalizzare senza tradire

- Prima l'alfabeto, poi la formula: dominio, costanti, funzioni (danno individui), predicati (danno vero o falso).
- «Ogni P è Q»: $\forall x.P(x) \Rightarrow Q(x)$. «Qualche P è Q»: $\exists x.P(x) \wedge Q(x)$.
- «Esattamente uno»: $\exists x.(P(x) \wedge \forall y.(P(y) \Rightarrow y = x))$. «Tranne uno»: come sopra, con la negazione al posto giusto.
- Un individuo senza nome nella frase è una variabile quantificata, nel punto in cui la frase lo introduce.
- Controllo finale: rileggi la formula ad alta voce e cerca un'interpretazione in cui la formula e la frase non sono d'accordo.

Verifica 4

a) In $(\exists x.p(x) \wedge (\forall x.q(x, y))) \vee r(x)$, quali occorrenze sono libere?

b) Formalizza «ogni numero pari ha un successore dispari» e dimmi se è un termine o una formula.

c) Perché $\forall x.p(x)$ è vera in $\mathcal{I}_2$ e falsa in $\mathcal{I}_3$?

Verità e leggi nel primo ordine

In breve
Come si decide se una formula con $\forall$ ed $\exists$ è vera? Serve sapere a chi si riferiscono le variabili: un **assegnamento**. Con questo strumento definiamo il valore di ogni formula, e poi raccogliamo le leggi e le regole che permettono di *dimostrare* con i quantificatori.

5.1 Il valore dei termini

Fissiamo un'interpretazione $\mathcal{I} = (D, \alpha)$. Un termine *chiuso* denota un elemento di D: una costante vale $\alpha(c)$; $g(t_1, \ldots, t_n)$ vale $\alpha(g)$ applicata ai valori dei t_i. Con $D = \mathbb{N}$, $\alpha(a) = 0$, $\alpha(s)(n) = n + 1$:

$$\alpha(s(s(a))) = \alpha(s)(\alpha(s)(\alpha(a))) = \alpha(s)(\alpha(s)(0)) = (0+1)+1 = 2.$$

$s(s(a))$ è una scrittura; 2 è il numero che essa denota.

Per un termine con variabili serve sapere a chi si riferiscono. Un **assegnamento** è una funzione ρ: $\mathcal{V} \to D$, per esempio $\rho = \{x \mapsto 0, y \mapsto 3, z \mapsto 1\}$. La notazione $\rho[d/x]$ indica l'assegnamento uguale a ρ tranne che manda x in d: $\rho[15/z] = \{x \mapsto 0, y \mapsto 3, z \mapsto 15\}$. Il valore $\alpha_\rho(t)$ aggiunge una sola regola: una variabile vale $\rho(x)$. Con $\rho = \{x \mapsto 2, y \mapsto 10\}$ si ha $\alpha_\rho(s(s(x))) = 4$ e $\alpha_\rho(s(s(y))) = 12$.

5.2 Il valore delle formule

Il valore di una formula dipende da $\mathcal{I}$ e da ρ; lo scriviamo $\mathcal{I}_\rho(\varphi)$. Le regole (S0)–(S4) sono quelle del capitolo 2: T vale t, F vale f, i connettivi si calcolano con le loro tabelle. Per una formula atomica, (S1): si calcolano i termini e si applica il predicato, $\mathcal{I}_\rho(p(t_1, \ldots, t_n)) = \alpha(p)(\alpha_\rho(t_1), \ldots, \alpha_\rho(t_n))$. Le novità sono i quantificatori:

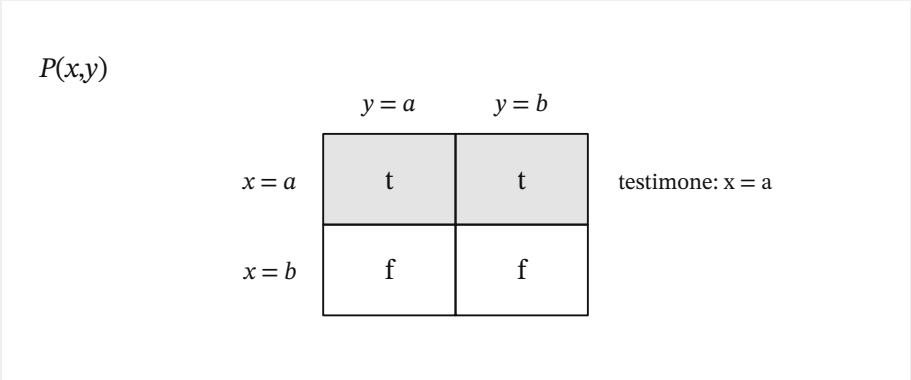
- **(S5)** $\mathcal{I}_\rho(\forall x.P) = \mathsf{t}$ se $\mathcal{I}_{\rho[d/x]}(P) = \mathsf{t}$ *per ogni* $d \in D$; altrimenti f.
- **(S6)** $\mathcal{I}_\rho(\exists x.P) = \mathsf{t}$ se $\mathcal{I}_{\rho[d/x]}(P) = \mathsf{t}$ *per almeno un* $d \in D$; altrimenti f.

Per valutare la portata, tipicamente aperta, si *prova* ogni elemento del dominio come valore di x. Per una formula **chiusa** l'assegnamento non conta (le slide lo enunciano senza dimostrarlo), e scriviamo semplicemente $\mathcal{I}(\varphi)$.

Immagine mentale · $\forall$ è un «e» lungo, $\exists$ è un «o» lungo
Su un dominio finito $\{a, b, c\}$, $\forall x.P(x)$ si comporta come $P(a) \wedge P(b) \wedge P(c)$ ed $\exists x.P(x)$ come $P(a) \vee P(b) \vee P(c)$. Per smontare un $\forall$ basta un elemento che fallisce; per un $\exists$ basta un elemento che funziona: un **testimone**.

Esercizio guidato · Un esistenziale con dentro un universale
$D = \{a, b\}$; Q vale t su entrambi; $P(x, y)$ vale t solo per (a, a) e (a, b). Stabilisci il valore di $\varphi_1 = \exists x.Q(x) \wedge (\forall y.P(x, y))$.

Idea. Per (S6) basta un testimone per x. Per il $\forall y$ interno bisogna invece provare tutti i valori di y.



La tabella di $P(x, y)$. La riga di a è tutta vera: a è il testimone per $\exists x.\forall y.P(x, y)$. La riga di b non funzionerebbe.

Conto. Proviamo $x \mapsto a$. $Q(a) = \mathsf{t}$. Per $\forall y.P(a, y)$: con $y \mapsto a$, $P(a, a) = \mathsf{t}$; con $y \mapsto b$, $P(a, b) = \mathsf{t}$. Quindi la portata vale $\mathsf{t} \wedge \mathsf{t} = \mathsf{t}$ e φ_1 è vera.

Controllo. Con $x \mapsto b$ avremmo $P(b, a) = \mathsf{f}$: per un $\exists$ basta però un solo testimone buono.

Allo stesso modo, con $D = \{a, b, c\}$ e i valori

x	a	b	c
P	t	t	f
Q	t	f	t
R	f	t	f

la formula $\forall x.P(x) \Rightarrow Q(x) \wedge R(x)$ è falsa: per $x = a$ la premessa è vera e $Q(a) \wedge R(a)$ è falsa. Un controesempio basta.

Le definizioni del capitolo 2 si estendono. Una formula chiusa è **valida** ($\vDash \varphi$) se è vera in ogni interpretazione, come $\forall x.p(x) \vee \neg p(x)$; **soddisfacibile** se è vera in almeno una, come $p(a)$ (basta $D = \mathbb{N}$, $\alpha(a) = 44$, p = «essere pari»); **insoddisfacibile** se non lo è mai, come $\exists x.p(x) \wedge \neg p(x)$. $\Gamma \vDash \varphi$ se ogni modello di Γ è modello di φ.

5.3 Dimostrare nella LPO

Una regola di inferenza si scrive $\frac{\delta_1, \ldots, \delta_k}{\delta}$: dalle premesse sopra si ricava la conclusione sotto; con $k = 0$ è un **assioma**. Una **dimostrazione** di φ da Γ è una sequenza di formule, ciascuna presa da Γ o ottenuta con una regola dalle precedenti, che finisce con φ; si scrive $\Gamma \vdash \varphi$. Un sistema è **corretto** se $\Gamma \vdash \varphi$ implica $\Gamma \vDash \varphi$, **completo** se vale il viceversa. Il calcolo proposizionale è corretto e completo; il sistema del corso per la LPO è corretto (sistemi completi esistono, come la deduzione naturale, ma non li vediamo).

Si usano tutte le leggi del capitolo 3, più leggi per i quantificatori, che sono *formule valide*. Se si usano anche ipotesi Γ_2, la dimostrazione garantisce $\Gamma_2 \vDash \varphi$, non la validità. Il **teorema di deduzione** giustifica il trucco di assumere la premessa: $\Gamma \vdash P \Rightarrow Q$ se e solo se $\Gamma, P \vdash Q$.

I principi di sostituzione restano validi, evitando catture; sotto un quantificatore la relazione usata deve valere per tutti i valori della variabile quantificata; per la polarità, $\forall$ ed $\exists$ *non* cambiano segno. Con l'ipotesi $P \Rightarrow Q$, il passo $(\forall x.P \vee R) \wedge (\exists x.\neg P) \Rightarrow (\forall x.Q \vee R) \wedge (\exists x.\neg P)$ è corretto (la prima P è positiva); sostituire anche la seconda P, che sta sotto $\neg$, sarebbe sbagliato.

5.4 Le leggi dei quantificatori

Nelle sostituzioni $P[t/x]$ si cambiano solo le occorrenze libere di x, evitando la cattura delle variabili di t: se necessario, prima si rinominano le variabili legate. Quando una legge istanzia un termine o nomina un individuo, si lavora su un dominio non vuoto. Il caso del dominio vuoto serve a capire quali ipotesi sono necessarie.

Nome	Legge
Elim. $\forall$, intro. $\exists$	$(\forall x.P) \Rightarrow P[t/x]$ $P[t/x] \Rightarrow (\exists x.P)$ (t termine, sostituite le occorrenze libere di x)
De Morgan	$\neg(\forall x.P) \equiv (\exists x.\neg P)$ $\neg(\exists x.P) \equiv (\forall x.\neg P)$
Annidamento	$\forall x.\forall y.P \equiv \forall y.\forall x.P$ $\exists x.\exists y.P \equiv \exists y.\exists x.P$
Costante	$(\forall x.P) \equiv P$ $(\exists x.P) \equiv P$ se x non occorre in P (dominio non vuoto)
Distribuzione	$(\forall x.P \wedge Q) \equiv (\forall x.P) \wedge (\forall x.Q)$ $(\exists x.P \vee Q) \equiv (\exists x.P) \vee (\exists x.Q)$
Solo implicazione	$(\forall x.P) \vee (\forall x.Q) \Rightarrow (\forall x.P \vee Q)$ $(\exists x.P \wedge Q) \Rightarrow (\exists x.P) \wedge (\exists x.Q)$
Distrib. con Q fisso	$(\forall x.P \wedge Q) \equiv (\forall x.P) \wedge Q$ $(\exists x.P \vee Q) \equiv (\exists x.P) \vee Q$ se x non occorre in Q (dominio non vuoto)
Indebolimento	$(\forall x.P) \Rightarrow (\forall x.P \vee Q)$ $(\exists x.P \wedge Q) \Rightarrow (\exists x.P)$, e analoghe
Dominio	$(\forall x\colon P \vee Q.R) \equiv (\forall x\colon P.R) \wedge (\forall x\colon Q.R)$ $(\exists x\colon P \vee Q.R) \equiv (\exists x\colon P.R) \vee (\exists x\colon Q.R)$

Esempi: dall'eliminazione di $\forall$, $(\forall x.\text{uomo}(x) \Rightarrow \text{mortale}(x)) \Rightarrow (\text{uomo}(\text{Socrate}) \Rightarrow \text{mortale}(\text{Socrate}))$; dall'introduzione di $\exists$, $\text{pari}(10) \wedge 10 > 2 \Rightarrow \exists x.\text{pari}(x) \wedge x > 2$. Per la riga «solo implicazione»: «ogni persona è maggiorenne o minorenne» è vera, ma non segue che tutte siano maggiorenni o tutte minorenni.

> **ATTENZIONE · Il dominio vuoto**
> Su un dominio vuoto non c'è nessun d che falsifichi la portata, quindi $\forall x.F$ è *vacuamente vera*, mentre F è falsa: la legge della costante fallirebbe. Per questo la legge richiede un dominio non vuoto (e dualmente $\exists x.T$ sarebbe falsa). Lo stesso accade con un quantificatore ristretto: se nessun elemento soddisfa P, $\forall x\colon P.Q$ è vera qualunque sia Q («tutti gli studenti con tre teste vanno al cinema» è vera), ed $\exists x\colon P.Q$ è falsa.

5.5 Uguaglianza

Nei linguaggi con $=$, $t_1 = t_2$ è vera quando i due termini denotano lo stesso elemento. La **legge di Leibniz** dice che oggetti uguali hanno le stesse proprietà, in tre forme equivalenti:

$$x = y \Rightarrow (P \equiv P[y/x]), \qquad x = y \wedge P \equiv x = y \wedge P[y/x], \qquad x = y \wedge P \Rightarrow P[y/x].$$

Le **leggi del singoletto** (con $x \neq y$ come nomi di variabile, e sostituzione senza cattura): $(\forall x.x = y \Rightarrow P) \equiv P[y/x]$ e $(\exists x.x = y \wedge P) \equiv P[y/x]$. L'idea: $x = y$ restringe il dominio a un solo elemento,

quindi quantificare equivale a sostituire. La seconda si dimostra così:

$$\begin{aligned} & \exists x.x = y \wedge P \\ \equiv\ & \exists x.x = y \wedge P[y/x] && \{\text{Leibniz}\} \\ \equiv\ & (\exists x.x = y) \wedge P[y/x] && \{\text{distribuzione: } x \text{ non occorre più libera in } P[y/x]\} \\ \equiv\ & P[y/x] && \{(\exists x.x = y) \equiv T, \text{ unità}\} \end{aligned}$$

5.6 Due regole per i quantificatori

Generalizzazione. Per dimostrare $\forall x.P$ si prende una costante *nuova* d (che non compare in Γ né in P) e si dimostra $P[d/x]$. d rappresenta un elemento generico, su cui non sappiamo nulla. Per esempio, da $\forall x.P \Rightarrow Q$ e $\forall x.P$: eliminando $\forall$ su d si ha $(P[d/x] \Rightarrow Q[d/x]) \wedge P[d/x]$, e per modus ponens $Q[d/x]$; per generalizzazione, $\forall x.Q$.

Skolemizzazione. Se tra le premesse c'è $\exists x.P$, per dimostrare qualunque Q si può aggiungere l'ipotesi $P[d/x]$ con d nuova, assente anche dalla conclusione Q: un elemento che soddisfa P esiste, e gli diamo un nome. Così si dimostra $(\forall x.P \Rightarrow Q) \wedge (\exists x.P) \Rightarrow (\exists x.Q)$:

Assumiamo le due premesse e chiamiamo d il testimone dell'esistenziale: aggiungiamo temporaneamente $P[d/x]$. Dall'universale otteniamo $P[d/x] \Rightarrow Q[d/x]$, quindi per modus ponens $Q[d/x]$, e per introduzione di $\exists$ concludiamo $\exists x.Q$. La conclusione non nomina d, perciò possiamo chiudere l'ipotesi sul testimone.

Attenzione. La scelta di d è una regola di prova, non la legge $(\exists x.P) \Rightarrow P[d/x]$ per una costante arbitraria: quest'ultima è falsa.

Con la generalizzazione si dimostra anche $(\exists y.\forall x.P) \Rightarrow (\forall x.\exists y.P)$: presa d nuova, $\exists y.\forall x.P(x, y) \Rightarrow \exists y.P(d, y)$ per eliminazione di $\forall$ (in posizione positiva dentro $\exists y$), e si generalizza su d.

> **Attenzione · $\forall\exists$ e $\exists\forall$ non si scambiano**
> Il viceversa è falso, e le slide non lo trattano. Esempio aggiunto: sui naturali $\forall x.\exists y.x < y$ è vera (ogni numero ha un numero più grande), mentre $\exists y.\forall x.x < y$ è falsa (non c'è un numero più grande di tutti). Nel primo caso y può dipendere da x, nel secondo no.

Officina della dimostrazione · 5. Lavorare con i quantificatori

- *Valutare* $\exists x.P$. Cerca un testimone. *Valutare* $\forall x.P$. Cerca un controesempio; se non c'è, devi controllare tutto il dominio.
- *Dimostrare* $\forall x.P$. Prendi una costante nuova d e dimostra $P[d/x]$ senza supporre nulla su d.
- *Usare* $\exists x.P$ *in premessa.* Dai un nome nuovo al testimone (skolemizzazione) e usa $P[d/x]$.
- *Usare* $\forall x.P$ *in premessa.* Istanzialo sul termine che ti serve.
- *Negare.* De Morgan scambia $\forall$ ed $\exists$ e porta la negazione dentro.

Verifica 5

a) Nega $\forall x.(p(x) \Rightarrow \exists y.q(x, y))$ portando la negazione fino agli atomi.

b) Perché nella generalizzazione la costante deve essere nuova?

c) Con $D = \{a, b\}$, $P(a, b) = P(b, a) = \mathsf{t}$ e $P(a, a) = P(b, b) = \mathsf{f}$: quanto valgono $\forall x.\exists y.P(x, y)$ e $\exists y.\forall x.P(x, y)$?

Capitolo 6

Stati, espressioni e comandi

In breve
Lasciamo la logica pura e torniamo ai programmi, con un piccolo linguaggio imperativo di soli interi e booleani. Diamo un significato preciso a due cose: le **espressioni**, che calcolano un valore leggendo lo stato, e i **comandi**, che cambiano lo stato. E scriviamo le specifiche come coppie di formule.

6.1 Perché una semantica formale

Di un linguaggio di programmazione si descrivono la **sintassi** (quali programmi sono scritti correttamente) e la **semantica** (che cosa significano). La sintassi si descrive con le grammatiche; per la semantica, in molte applicazioni non basta che un programma «sembri funzionare»: per centrali nucleari, apparecchiature mediche o carte di credito bisogna *dimostrarne* le proprietà. Tre approcci:

Approccio	Idea
operazionale	si descrivono gli stati e le transizioni da uno stato all'altro
denotazionale	a ogni costrutto si associa una funzione matematica
assiomatico	si annota il programma con asserzioni e se ne dimostra la correttezza con regole di inferenza (Hoare 1969, Dijkstra 1976)

Vedremo una semantica denotazionale in questo capitolo e quella assiomatica nei successivi. I valori sono Val$=\mathbb{Z}\cup\mathrm{Bool}$.

6.2 Stati e specifiche

Uno **stato** associa a ogni identificatore al più un valore: è una funzione parziale $\sigma\colon \mathrm{Ide}\to\mathrm{Val}$, come $\sigma=\{x\mapsto 10, y\mapsto 0\}$. È la stessa struttura dell'assegnamento ρ del capitolo 5, con gli identificatori al posto delle variabili.

Una **specifica** è una coppia di **asserzioni** (formule con variabili libere): proprietà dello stato iniziale $\longrightarrow$ proprietà dello stato finale. Lo stato fa la parte dell'assegnamento: σ **soddisfa** P, $\sigma\vDash P$, se P è vera quando le variabili prendono i valori di σ (con l'interpretazione usuale dei numeri). Per il massimo di due naturali:

$$x\geq 0\wedge y\geq 0 \quad\longrightarrow\quad z=x\max y.$$

Attenzione · Una specifica troppo debole
L'«algoritmo» «metti 2, 50, 50 in x, y, z» la rispetta: da $\{x\mapsto 20, y\mapsto 3\}$ arriva a uno stato con $z=50=2\max 50$. Ma non è ciò che volevamo: il massimo doveva essere quello dei valori *iniziali*, e x, y non dovevano cambiare.

Per parlare dei valori iniziali si usano **variabili di specifica** (maiuscole), che a differenza delle variabili di programma nessun comando può modificare:

$$x=X\wedge y=Y\wedge X\geq 0\wedge Y\geq 0 \quad\longrightarrow\quad z=X\max Y.$$

Ora da $\{x \mapsto 20, y \mapsto 3\}$ si ha $X = 20$, $Y = 3$ e serve $z = 20$. Un algoritmo corretto: «se $x < y$ assegna a z il valore di y, altrimenti quello di x». Altri esempi:

Problema	Specifica
raddoppiare un naturale	$x = A \wedge A > 0 \longrightarrow x = 2 \cdot A$
valore assoluto	$x = A \longrightarrow x = \lvert A \rvert$
quoziente e resto	$x = X \wedge y = Y \wedge X \geq 0 \wedge Y > 0 \longrightarrow X = q \cdot Y + r \wedge 0 \leq r < Y$

Spesso si abbrevia scrivendo «*con x, y costanti*»: il programma deve lasciare x e y come li ha trovati, e le variabili di specifica non servono. Per esempio $x \geq 0 \wedge y > 0 \longrightarrow x = q \cdot y + r \wedge 0 \leq r < y$ con x, y costanti. A volte sono però inevitabili: in $x = A \wedge A > 0 \longrightarrow x = 2 \cdot A$ è proprio x a cambiare.

6.3 Espressioni

Le **espressioni** sono costanti intere (`0`, `125`, `-5`), costanti booleane (`true`, `false`, nomi riservati), identificatori, e le loro combinazioni con `not`, con $+, -, ,$ div,mod e con $=, \neq, <, >, \leq, \geq$, and, or. Valgono le precedenze usuali; le parentesi disambiguano: `(8+5)*3` contro `8+(5*3)`.

`(x + 5) and (x > y)` è scritta bene, ma non ha senso: `and` vuole due booleani. Lo scopre il **controllo dei tipi**, detto **semantica statica** perché guarda solo la struttura, senza calcolare. D'ora in poi le espressioni sono sempre staticamente corrette.

Il valore di un'espressione in uno stato è dato dalla funzione $\mathcal{E}$: Exp×State→Val, definita sulla struttura:

$$\mathcal{E}(\mathtt{n}, \sigma) = \mathbf{n}, \qquad \mathcal{E}(x, \sigma) = \sigma(x), \qquad \mathcal{E}(\mathtt{not}\ E, \sigma) = \neg \mathcal{E}(E, \sigma), \qquad \mathcal{E}(E \text{ op } E', \sigma) = \mathcal{E}(E, \sigma)\ \mathbf{op}\ \mathcal{E}(E', \sigma).$$

Il grassetto indica l'oggetto matematico che interpreta il simbolo (la costante `5` denota il numero 5; `and` denota $\wedge$). Anche per espressioni corrette, $\mathcal{E}$ è una funzione **parziale**: $\mathcal{E}(\mathtt{x\ div\ 0}, \sigma) = \sigma(x)\ \mathbf{div}\ 0 = \perp$, il valore indefinito.

6.4 Comandi

I **comandi** modificano lo stato.

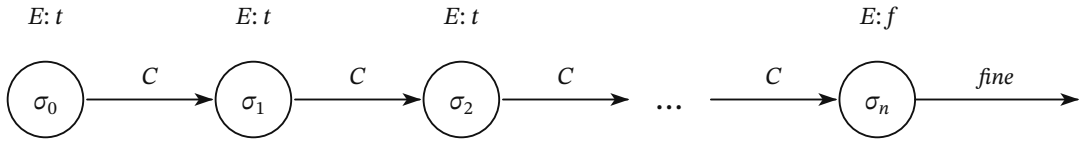
Comando	Sintassi
vuoto	`skip`
assegnamento	`x := E`
sequenza	`C ; C'`
condizionale	`if E then C else C' fi` (guardia E, ramo then, ramo else)
iterativo	`while E do C endw` (guardia E, corpo C)

La semantica è la funzione parziale $\mathcal{C}$: Com×State→State. Scriviamo $\sigma[v/x]$ per lo stato uguale a σ tranne che in x, dove vale v.

- $\mathcal{C}(\mathtt{skip}, \sigma) = \sigma$.
- $\mathcal{C}(\mathtt{x\ :=\ E}, \sigma) = \sigma[\mathcal{E}(E, \sigma)/x]$: si calcola E nello stato *vecchio*, poi si cambia x.
- $\mathcal{C}(C; C', \sigma) = \mathcal{C}(C', \mathcal{C}(C, \sigma))$: si esegue C, poi C' dallo stato raggiunto.
- Il condizionale esegue C se $\mathcal{E}(E, \sigma) = \mathtt{t}$, C' se $\mathcal{E}(E, \sigma) = \mathtt{f}$.
- $\mathcal{C}(\mathtt{while\ E\ do\ C\ endw}, \sigma)$ è σ se la guardia è falsa; altrimenti è $\mathcal{C}(\mathtt{while\ E\ do\ C\ endw}, \mathcal{C}(C, \sigma))$. La definizione è *ricorsiva*.

Per esempio, `if x > 0 then skip else x := -x fi` lascia invariato $\{x \mapsto 25\}$ e porta $\{x \mapsto -25\}$ in $\{x \mapsto 25\}$. E `x := y; x := x + 1` porta $\{x \mapsto 10, y \mapsto 0\}$ prima in $\{x \mapsto 0, y \mapsto 0\}$, poi in $\{x \mapsto 1, y \mapsto 0\}$.

Il `while` esegue il corpo più volte: $C_1; C_2; C_3; \ldots$, tutte copie di C. Se la sequenza è finita, si ferma al primo stato in cui la guardia è falsa (anche subito, con zero giri); se è infinita, la guardia resta vera per sempre e $\mathcal{C} = \bot$. Anche `x := y div 0` dà $\bot$.



Il `while` srotolato: si controlla la guardia, si esegue il corpo, si ricontrolla. Ci si ferma al primo stato σ_n in cui la guardia è falsa.

`while x >= y do x := x - 1 endw` termina sempre: x scende di 1 e y resta ferma. `while x >= y do x := x + 1 endw` può non terminare: se all'inizio $x \geq y$, la guardia resta vera per sempre.

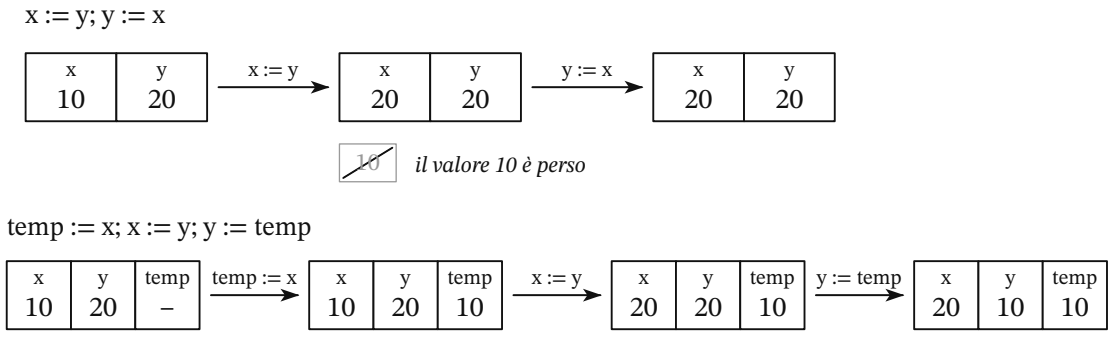
6.5 Primi algoritmi

Valore assoluto. $\{\text{numero} \mapsto A\} \longrightarrow \{\text{risultato} \mapsto |A|\}$:

```
if numero > 0 then risultato := numero else risultato := -numero fi
```

Scambio. $\{x \mapsto A, y \mapsto B\} \longrightarrow \{x \mapsto B, y \mapsto A\}$. La prima idea, `x := y; y := x`, è sbagliata: calcolando, $\mathcal{C}(\texttt{x := y; y := x}, \sigma) = \mathcal{C}(\texttt{y := x}, \sigma[B/x]) = \{x \mapsto B, y \mapsto B\}$. Il valore di x è perso. Serve un appoggio:

```
temp := x; x := y; y := temp
```



Perché serve `temp`. L'assegnamento è *distruttivo* (il vecchio valore viene sovrascritto) e i comandi sono eseguiti in sequenza: chi viene sovrascritto per primo deve essere messo da parte.

Ordinare due numeri. $\{\text{n1} \mapsto A, \text{n2} \mapsto B\} \longrightarrow \{\text{n1} \mapsto \max(A, B), \text{n2} \mapsto \min(A, B)\}$:

```
if n1 < n2 then temp := n1; n1 := n2; n2 := temp else skip fi
```

Esercizio guidato · Elevamento a potenza

Specifica: $\{\text{base}\mapsto A, \text{esp}\mapsto B\}$ con $A>0$, $B\geq 0 \longrightarrow \{\text{ris}\mapsto A^B\}$, seguendo $A^0=1$ e $A^B=A\cdot A\cdots A$ (B volte). Esegui il programma con $A=2$, $B=3$.

```
ris := 1;
while esp > 0 do
    ris := ris * base;
    esp := esp - 1
endw
```

Idea. Una riga per ogni controllo della guardia; a destra di `:=` si usano i valori vecchi.

Conto.

giro	`ris`	`esp`	guardia `esp > 0`
inizio	1	3	t
1	2	2	t
2	4	1	t
3	8	0	f

Controllo. ris$=8=2^3$. Con $B=0$ il corpo non si esegue mai e ris$=1=A^0$: anche il caso limite è giusto.

Officina della dimostrazione · 6. Tracciare uno stato

Per capire che cosa fa un programma, eseguilo a mano su un esempio piccolo.

- Una colonna per ogni variabile, una riga dopo ogni comando (o dopo ogni giro).
- In `x := E` calcola E con i valori della riga precedente, *poi* scrivi il nuovo x.
- Per ogni `while` annota il valore della guardia a ogni controllo: l'ultimo è `f`.
- Prova anche i casi limite (zero giri, valori uguali, zero).

Una traccia non dimostra nulla «per ogni stato», ma trova molti errori e suggerisce che cosa resta vero a ogni giro: l'invariante del capitolo 8.

Verifica 6

a) In che stato porta `x := x + y; y := x - y; x := x - y` lo stato $\{x\mapsto 3, y\mapsto 7\}$?

b) Perché $x\geq 0 \longrightarrow x=2\cdot x$ non va bene come specifica del raddoppio?

c) `while x ≠ 0 do x := x - 2 endw` termina partendo da $x=5$?

Triple di Hoare

In breve
Una tripla di Hoare $\{Q\}\ C\ \{R\}$ collega un programma alle sue specifiche: se si parte da uno stato che soddisfa Q, il comando C termina con successo in uno stato che soddisfa R. Impariamo a leggere una tripla in tre modi (per capire un comando, per scrivere un comando, per dimostrarne la correttezza) e a costruire le prove con regole di inferenza.

7.1 Il significato di una tripla

Una **tripla di Hoare** è una terna $\{Q\}\ C\ \{R\}$: Q è la **precondizione**, R la **postcondizione**, C un comando. La tripla è *soddisfatta* se, da ogni stato $\sigma \vDash Q$, l'esecuzione di C *termina con successo* in uno stato $\sigma' \vDash R$. « Con successo» vuol dire senza errori: un assegnamento che divide per zero non termina con successo.

$$\{x > 1\}\ x := x + 1\ \{x > 2\}$$

Questa tripla è soddisfatta: da ogni stato con $x > 1$ l'assegnamento porta a uno stato con $x > 2$.

Servono alcune notazioni. $\text{free}(P)$ è l'insieme delle variabili libere di P: per esempio $\text{free}(x > y \wedge z \leq 1) = \{x, y, z\}$. P^σ è l'asserzione P in cui ogni variabile libera è sostituita dal suo valore in σ, e $\sigma \vDash P$ vale se P^σ è vera. Con $\sigma = \{x \mapsto 18, y \mapsto \mathtt{t}, z \mapsto -8\}$ e $P \equiv (x > 0 \wedge z < 0)$ si ha $P^\sigma \equiv (18 > 0 \wedge -8 < 0) \equiv \mathtt{t}$. Infine $\{P\}$ indica l'insieme degli stati che soddisfano P.

Tre proprietà, usate senza dimostrazione (le slide le omettono; seguono dalle definizioni):

- **(P1)** Se $\sigma(x) = \sigma'(x)$ per ogni $x \in \text{free}(E)$, allora $\mathcal{E}(E, \sigma) = \mathcal{E}(E, \sigma')$.
- **(P2)** Se $\sigma(x) = \sigma'(x)$ per ogni $x \in \text{free}(P)$, allora $\sigma \vDash P$ se e solo se $\sigma' \vDash P$.
- **(P3)** $\sigma[\mathcal{E}(E, \sigma)/x] \vDash P$, con E definita e sostituzione senza cattura, se e solo se $\sigma \vDash P[E/x]$.

(P3) dice che assegnare a x il valore di E e poi valutare P equivale a valutare, nello stato di partenza, P con E al posto di x. Due esempi:

$$\sigma[\mathcal{E}(z + 1, \sigma)/x] \vDash (x > y) \text{ se e solo se } \sigma \vDash (x > y)[z + 1/x]$$

$$\sigma[\mathcal{E}(x + y, \sigma)/x] \vDash (x = z) \text{ se e solo se } \sigma \vDash (x = z)[x + y/x]$$

Nel primo, il valore di x dopo l'assegnamento è $\sigma(z) + 1$, e $\sigma(z) + 1 > \sigma(y)$ vale se e solo se $z + 1 > y$. Nel secondo il valore è $\sigma(x) + \sigma(y)$, e l'uguaglianza con $\sigma(z)$ è proprio $x + y = z$.

7.2 Tre usi di una tripla

La stessa tripla si può leggere in tre modi.

- **Semantica.** Dati Q e C, si cerca R che descriva il comportamento di C. Con $C \equiv$ `if x < 10 then x := 1 else x := 2 fi` e $Q \equiv (x > 10)$, la postcondizione $R \equiv (x = 2)$ descrive il comando: la guardia è falsa, si esegue il ramo `else` e si ottiene $x = 2$.

- **Specifica.** Dati Q e R, si cerca un C che renda vera la tripla: è scrivere un programma che realizza la specifica. Con $Q \equiv (x > 0)$ e $R \equiv (y = 2 \cdot x)$ vanno bene `y := x * 2`, `y := x + x`, `y := x; y := y * 2`. Ma va bene anche `y := 10; x := 5`: nello stato finale $y = 10 = 2 \cdot 5$. Non è ciò che volevamo, perché il programma ha cambiato x. Per escluderlo servono le variabili di specifica del capitolo 6.

- **Correttezza.** Dati C, Q e R, si dimostra che la tripla è soddisfatta. È l'uso che faremo in questo corso. Per esempio, il programma seguente soddisfa la tripla con $Q \equiv (x \geq 0 \wedge y > 0)$ e $R \equiv (x = q \cdot y + r \wedge 0 \leq r < y)$:

```
q := 0;
r := x;
while r >= y do
    q := q + 1;
    r := r - y
endw
```

7.3 Regole per pre- e postcondizione

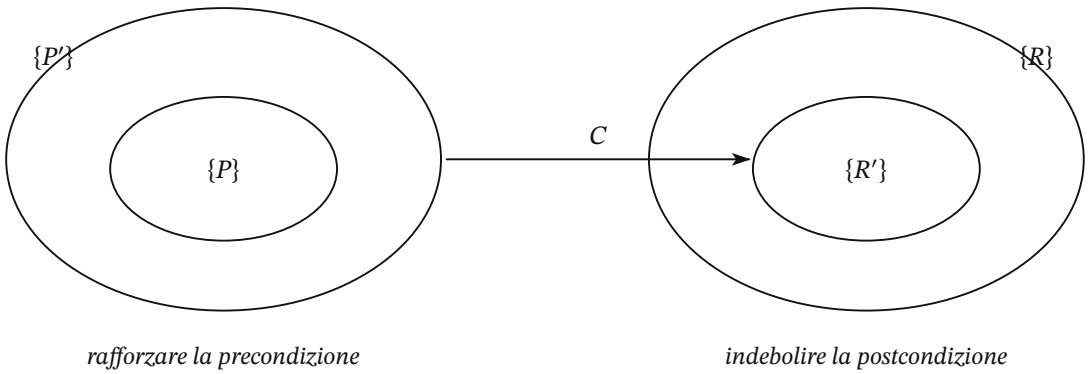
Una prova è una sequenza di regole di inferenza: sopra la linea le premesse, sotto la conclusione. Le prime due regole indeboliscono una tripla: si può partire da un insieme più piccolo di stati e arrivare a uno più grande.

$$\frac{P \Rightarrow P' \qquad \{P'\}\ C\ \{R\}}{\{P\}\ C\ \{R\}} \quad \text{(pre)}$$

$$\frac{\{P\}\ C\ \{R'\} \qquad R' \Rightarrow R}{\{P\}\ C\ \{R\}} \quad \text{(post)}$$

Le due insieme danno la regola (pre-post):

$$\frac{P \Rightarrow P' \qquad \{P'\}\ C\ \{R'\} \qquad R' \Rightarrow R}{\{P\}\ C\ \{R\}} \quad \text{(pre-post)}$$



Rafforzare la precondizione restringe l'insieme di partenza; indebolire la postcondizione allarga quello di arrivo. La tripla resta valida.

Lo `skip` non cambia lo stato, quindi ha un assioma senza premesse:

$$\{P\}\ \texttt{skip}\ \{P\} \quad \text{(skip)}$$

7.4 Quando un'espressione è definita

Un'espressione può non avere valore per due motivi: la tipizzazione è scorretta, come in `5 + true`, ma questo caso è escluso da un controllo dei tipi preliminare; oppure il valore è indefinito, come in `x := 0; y := k div x`. Per il secondo caso introduciamo una funzione def che, dato E, restituisce un'asserzione tale che $\sigma \vDash \mathrm{def}(E)$ se e solo se $\mathcal{E}(E,\sigma)$ è definito.

- $\mathrm{def}(c) = \mathrm{def}(x) = \mathtt{t}$, per costanti c e identificatori x.
- $\mathrm{def}(E \mathrel{\mathrm{op}} E') = \mathrm{def}(E) \wedge \mathrm{def}(E')$, per $\mathrm{op} \in \{+, -, ,, =, \neq, <, >, \leq, \geq, \mathtt{and}, \mathtt{or}\}$.
- $\mathrm{def}(E \,\mathrm{div}\, E') = \mathrm{def}(E) \wedge \mathrm{def}(E') \wedge E' \neq 0$, e lo stesso per mod .
- $\mathrm{def}(\mathtt{not} E) = \mathrm{def}(E)$.

Come nelle regole delle slide, qui gli identificatori letti si assumono presenti nello stato e del tipo dichiarato. Senza questa ipotesi, anche la lettura di un nome assente sarebbe indefinita. Le sole ulteriori fonti di indefinitezza sono quindi div e mod con secondo argomento nullo. Per esempio $\mathrm{def}(x \,\mathrm{div}\, (z - y))$ vale $z \neq y$:

$$
\begin{array}{ll}
\quad \mathrm{def}(x \,\mathrm{div}\, (z-y)) & \\
\equiv \mathrm{def}(x) \wedge \mathrm{def}(z-y) \wedge z - y \neq 0 & \{\text{definizione di def}\} \\
\equiv \mathtt{t} \wedge \mathrm{def}(z-y) \wedge z \neq y & \{\mathrm{def}(x) \equiv \mathtt{t};\ z - y \neq 0 \text{ è } z \neq y\} \\
\equiv \mathrm{def}(z) \wedge \mathrm{def}(y) \wedge z \neq y & \{\text{def di una differenza, unità}\} \\
\equiv z \neq y & \{\mathrm{def}(z) \equiv \mathrm{def}(y) \equiv \mathtt{t}, \text{ unità}\}
\end{array}
$$

7.5 Assegnamento

L'assioma dell'assegnamento si usa *all'indietro*, dalla postcondizione alla precondizione:

$$\{\mathrm{def}(E) \wedge P[E/x]\}\ x := E\ \{P\} \quad \text{(ass)}$$

dove $P[E/x]$ è P con E al posto di x. L'intuizione: perché P valga dopo l'assegnamento, deve valere prima la stessa P con E al posto di x; e E deve avere un valore nello stato di partenza.

Per esempio, con $P \equiv (x > y)$:

$$\{x + 2 > y\}\ x := x + 2\ \{x > y\}$$

perché $(x > y)[x + 2/x] \equiv (x + 2 > y)$ e $\mathrm{def}(x + 2) \equiv \mathtt{t}$. Con una costante, $\{\mathtt{t}\}\ x := 5\ \{x = 5\}$, perché $\mathrm{def}(5) \wedge (5 = 5) \equiv \mathtt{t}$.

Per *verificare* un assegnamento, si combina l'assioma con la regola (pre). Si ottiene una regola in cui resta solo un'implicazione:

$$\frac{R \Rightarrow \mathrm{def}(E) \wedge P[E/x]}{\{R\}\ x := E\ \{P\}} \quad \text{(ass), verifica}$$

7.6 Sequenza

La regola della sequenza unisce due triple con un'asserzione intermedia R, che fa da ponte:

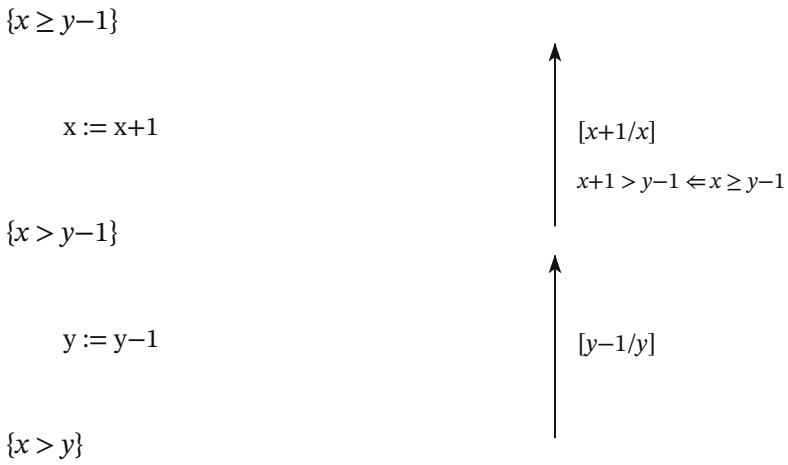
$$\frac{\{P\}\ C\ \{R\} \qquad \{R\}\ C'\ \{Q\}}{\{P\}\ C; C'\ \{Q\}} \quad \text{(seq)}$$

Esercizio guidato · Due assegnamenti in sequenza
Verifica $\{x \geq y-1\}\ x{:=}x+1; y{:=}y-1\ \{x > y\}$.

Idea. Il comando finale è `y := y - 1`. Si calcola all'indietro la precondizione di ciascun comando, e si usa il risultato come intermedia R.

Conto. Per `y := y - 1` con postcondizione $x > y$: $(x > y)[y-1\,/\,y] \equiv (x > y-1)$, quindi $R \equiv (x > y-1)$. Per `x := x + 1` con postcondizione R: $R[x+1\,/\,x] \equiv (x+1 > y-1) \equiv (x > y-2)$. Basta quindi $x \geq y-1 \longrightarrow x > y-2$, che vale perché $x \geq y-1 > y-2$.

Controllo. Con $x=4$, $y=5$: la precondizione è vera ($4 \geq 4$). Dopo `x := x + 1` si ha $x=5$, poi `y := y - 1` dà $y=4$, e $5>4$.



Il calcolo all'indietro: la postcondizione scende dal fondo del programma verso l'inizio, un comando alla volta, e l'ultima asserzione è la precondizione richiesta.

7.7 Condizionale

$$\frac{P \Rightarrow \mathrm{def}(E) \qquad \{P \wedge E\}\ C\ \{Q\} \qquad \{P \wedge \neg E\}\ C'\ \{Q\}}{\{P\}\ \texttt{if E then C else C' fi}\ \{Q\}}\ \text{(cond)}$$

La prima premessa garantisce che la guardia sia definita; le altre due trattano i rami, ciascuno con la sua ipotesi sulla guardia.

Esercizio guidato · Il massimo di due valori
Verifica $\{m=0\}$ `if x < y then m := y else m := x fi` $\{m = \max(x,y)\}$.

Idea. Il ramo `then` deve dare il massimo quando $x < y$, il ramo `else` quando $x \geq y$.

Conto. Per (cond) servono tre obblighi. (1) $m=0 \longrightarrow \mathrm{def}(x<y)$: vera, perché $\mathrm{def}(x<y) \equiv \mathrm{def}(x) \wedge \mathrm{def}(y) \equiv \mathtt{t}$. (2) Per il ramo `then`, con (ass) basta $m=0 \wedge x<y \longrightarrow y = \max(x,y)$, vera perché il massimo è y quando $x<y$. (3) Per il ramo `else`, basta $m=0 \wedge x \geq y \longrightarrow x = \max(x,y)$, vera per lo stesso motivo.

Controllo. L'ipotesi $m=0$ non serve: con $x=3$, $y=7$ il ramo `then` dà $m=7=\max(3,7)$; con $x=7$, $y=3$ il ramo `else` dà $m=7$.

7.8 Trovare l'espressione mancante

Determiniamo E in modo che sia vera la tripla

$$\{x = N \wedge y = M\}\ t{:=}E; x{:=}y; y{:=}t\ \{x = M \wedge y = N\}$$

dove M e N sono variabili di specifica. Con (seq) servono R_1 e R_2, e si procede all'indietro con (ass):

- Dal terzo comando, $R_2 \equiv (x = M \wedge t = N)$, perché $(x = M \wedge y = N)[t/y] \equiv (x = M \wedge t = N)$.
- Dal secondo, $R_1 \equiv (y = M \wedge t = N)$, perché $(x = M \wedge t = N)[y/x] \equiv (y = M \wedge t = N)$.
- Dal primo, resta da trovare E tale che $x = N \wedge y = M \longrightarrow \mathrm{def}(E) \wedge (y = M \wedge E = N)$.

Quindi $E \equiv x$: nello stato iniziale $x = N$.

> **ATTENZIONE · Non è E=N**
> Il valore cercato è N, ma N non può comparire in un comando: è una variabile di specifica. Il comando deve leggere x, che nello stato iniziale vale proprio N.

Ora due triple che *non* sono soddisfatte. Perché?

- $\{z = 5 \wedge y = 7 \wedge x = 3\}$ `x := 0; y := z div x` $\{z > 4\}$. Dopo `x := 0` il comando `y := z div x` divide per zero: l'esecuzione non termina con successo, anche se $z > 4$ resterebbe vera. Infatti $\mathrm{def}(z \,d\, i \,v\, x) \equiv x \neq 0$, e dopo `x := 0` questo diventa $0 \neq 0$, falso.
- $\{x = N \wedge y = M\}$ $x{:=}y; y{:=}x$ $\{x = M \wedge y = N\}$. Lo stato finale ha $x = M$ e $y = M$: la postcondizione vale solo se $M = N$. È lo scambio sbagliato visto nel capitolo 6.

Un ultimo esempio di condizionale in sequenza. Verifichiamo $\{x = 5\}$ $x{:=}3;$ `if (x = 3) then y := 7 else y := 5 fi` $\{x = 3 \wedge y = 7\}$. Per (seq) il candidato naturale è $R \equiv (x = 3)$. Il primo passo, $\{x = 5\}$ $x{:=}3$ $\{x = 3\}$, vale perché $x = 5 \longrightarrow \mathrm{def}(3) \wedge 3 = 3$. Per il secondo, con (cond): $x = 3 \longrightarrow \mathrm{def}(x = 3) \equiv \mathtt{t}$; nel ramo `then`, $x = 3 \wedge x = 3 \longrightarrow \mathrm{def}(7) \wedge x = 3 \wedge 7 = 7$, vera; nel ramo `else`, la premessa $x = 3 \wedge \neg(x = 3)$ è falsa, quindi l'implicazione è vera.

7.9 Rafforzare la precondizione

Si verifichi

$$\{x \geq 0 \wedge y \geq 0\} \texttt{ if x div y > 0 then z := x else z := y fi } \{z = x \max y\}$$

La regola (cond) richiede $x \geq 0 \wedge y \geq 0 \longrightarrow \mathrm{def}(x \text{ div } y > 0)$, cioè $y \neq 0$. Ma la precondizione non lo garantisce: con $y = 0$ la guardia non è definita. Per avere una tripla soddisfatta bisogna *rafforzare la precondizione*:

$$\{x \geq 0 \wedge y > 0\} \texttt{ if x div y > 0 then z := x else z := y fi } \{z = x \max y\}$$

Con $x \geq 0$ e $y > 0$, la guardia $x \text{ div } y > 0$ equivale a $x \geq y$. Nel ramo `then` serve $x \geq y \longrightarrow x = \max(x, y)$; nel ramo `else`, con $x < y$, serve $y = \max(x, y)$. Entrambe sono vere.

7.10 Correttezza totale e parziale

Finora una tripla è di **correttezza totale**, $\vDash_{tot} \{Q\}\ C\ \{R\}$: da ogni stato di Q, C termina con successo in uno stato di R. La terminazione è inclusa grazie a def. Con **correttezza parziale**, $\vDash_{par} \{Q\}\ C\ \{R\}$, si richiede solo che, *se* C termina, lo faccia in uno stato di R. Per questa nozione si possono indebolire le regole, ignorando def:

$$\{Q[E/x]\}\ x{:=}E\ \{Q\} \quad \text{(ass, parziale)} \qquad \frac{P \Rightarrow Q[E/x]}{\{P\}\ x{:=}E\ \{Q\}} \quad \text{(ass, parziale)}$$

$$\frac{\{P \wedge E\}\ C\ \{Q\} \qquad \{P \wedge \neg E\}\ C'\ \{Q\}}{\{P\}\ \texttt{if E then C else C' fi}\ \{Q\}} \quad \text{(cond, parziale)}$$

Officina della dimostrazione · 7. Calcolare all'indietro

Data una postcondizione e un programma, si cerca la precondizione che il programma richiede. Oppure si dimostra una tripla data.

- Parti dalla postcondizione e scorri i comandi dal fondo verso l'inizio.
- Per ogni assegnamento `x := E`, sostituisci E a x nell'asserzione e aggiungi $\mathrm{def}(E)$.
- Per un condizionale, la postcondizione resta la stessa nei due rami; le guardie danno $P \wedge E$ e $P \wedge \neg E$.
- Alla fine confronta la precondizione ottenuta con quella data: serve un'implicazione.

Il calcolo all'indietro torna nel capitolo 8, dove la postcondizione di un ciclo si costruisce con un invariante.

Verifica 7

a) Perché la tripla $\{x = 5\}\ x := x + 1\ \{x = 5\}$ non è soddisfatta?

b) Calcola con (ass) la precondizione di `y := x * 2` rispetto alla postcondizione $y > 10$.

c) Perché $\{x \geq 0\}$ `y := 1 div x` $\{y = 1\}$ non è soddisfatta?

Cicli: invarianti e terminazione

In breve
Un ciclo può girare un numero di volte che non conosciamo prima di eseguirlo. Per ragionarne cerchiamo un **invariante**, un'asserzione che resta vera a ogni giro, e per la terminazione una **funzione di terminazione**, che ogni giro fa diminuire. Con queste due idee si costruiscono le prove di correttezza dei programmi con `while`.

8.1 Il problema del ciclo

Il comando `while E do C endw` esegue copie del corpo C finché la guardia E è vera, ma non sappiamo in anticipo quante. Se i giri fossero k fissi, basterebbe concatenare k volte la regola (seq): $\{P\}C\{P_1\}$, $\{P_1\}C\{P_2\}$, ..., $\{P_{k-1}\}C\{R\}$. Ma k non è noto.

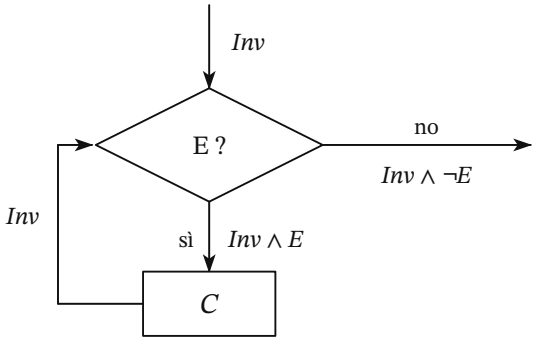
Per questo si cerca un'unica asserzione Inv, l'**invariante**, che sostituisca tutte le intermedie. Ragionando sulla correttezza parziale, si può concludere che il ciclo ha la forma $\{\text{Inv}\}$ `while E do C endw` $\{\text{Inv}\wedge\neg E\}$. Le regole sono due. La prima:

$$\frac{\{\text{Inv}\wedge E\}\ C\ \{\text{Inv}\}}{\{\text{Inv}\}\ \texttt{while E do C endw}\ \{\text{Inv}\wedge\neg E\}} \quad (1)$$

Non serve che Inv valga dopo ogni singolo comando del corpo: basta che sia ristabilito prima della successiva valutazione della guardia. La seconda regola parte da una tripla generica con P e R qualunque, e rafforza le premesse con (pre-post):

$$\frac{P\Rightarrow\text{Inv} \qquad \{\text{Inv}\wedge E\}\ C\ \{\text{Inv}\} \qquad \text{Inv}\wedge\neg E\Rightarrow R}{\{P\}\ \texttt{while E do C endw}\ \{R\}} \quad (1\text{-bis})$$

La (1) e la (1-bis) danno solo la correttezza *parziale*: non dicono nulla sulla terminazione.



L'invariante è una zona di stati che il ciclo non abbandona. Ogni giro parte dalla zona e vi ritorna; all'uscita la guardia è falsa, e la zona più $\neg E$ dà la postcondizione.

Un invariante deve essere *significativo*. `t` è invariante di ogni ciclo, perché ogni stato lo soddisfa, ma non dice nulla del programma.

8.2 Correttezza totale

Per la terminazione servono tre cose: la guardia deve essere definita, ogni giro deve terminare con successo, e il numero di giri deve essere finito. La guardia è coperta da def(E); per il terzo punto

si usa una **funzione di terminazione** expt, a valori interi, con due ipotesi:

$$\text{Inv}\Rightarrow\text{expt}\geq 0 \qquad \text{e} \qquad \{\text{Inv}\wedge E\wedge \text{expt}=V\}\ C\ \{\text{expt}<V\}$$

dove V è una variabile di specifica, che il corpo non modifica e che memorizza il valore di expt prima del giro. La prima dice che expt non scende sotto zero; la seconda che ogni giro la fa diminuire strettamente. Non esiste una successione infinita strettamente decrescente di naturali: quindi il ciclo non può girare per sempre.

Servono entrambe le ipotesi. Per `while x >0 do x := x+1 endw` con expt$=-x$, la diminuzione c'è, ma $-x$ non è non negativa: da $x=1$ il ciclo non termina. Viceversa, per `while x ≠ 0 do x := x+1 endw` con Inv$\equiv(x\geq 0)$ e expt$=x$, la non negatività c'è, ma x non diminuisce: da $x=1$ il ciclo non termina.

La regola per la correttezza totale è:

$$\frac{\begin{array}{c}\text{Inv}\Rightarrow\text{expt}\geq 0\\ \{\text{Inv}\wedge E\wedge\text{expt}=V\}\ C\ \{\text{expt}<V\}\\ \{\text{Inv}\wedge E\}\ C\ \{\text{Inv}\wedge\text{def}(E)\}\end{array}}{\{\text{Inv}\wedge\text{def}(E)\}\ \texttt{while E do C endw}\ \{\text{Inv}\wedge\neg E\}}\quad\text{(while)}$$

Per una specifica $\{P\}$ `while E do C endw` $\{Q\}$ si verificano cinque obblighi:

Obbligo	Che cosa bisogna dimostrare
Ingresso	$P\Rightarrow\text{Inv}\wedge\text{def}(E)$
Invarianza	$\{\text{Inv}\wedge E\}\ C\ \{\text{Inv}\wedge\text{def}(E)\}$
Terminazione	$\text{Inv}\Rightarrow\text{expt}\geq 0$
Progresso	$\{\text{Inv}\wedge E\wedge\text{expt}=V\}\ C\ \{\text{expt}<V\}$
Uscita	$\text{Inv}\wedge\neg E\Rightarrow Q$

Spesso P non basta a stabilire l'invariante. Si premette allora un comando di inizializzazione CI con $\{P\}$ CI $\{\text{Inv}\wedge\text{def}(E)\}$, e per (seq) si conclude $\{P\}$ CI; `while E do C endw` $\{Q\}$.

Esercizio guidato · La somma dei primi interi

Dimostra la specifica $n>0\longrightarrow s=\sum_{i=0}^{n-1} i$, con n non modificato, per questo programma:

```
x := 0; s := 0;
while x < n do
    s := s + x;
    x := x + 1
endw
```

Idea. s deve contenere la somma degli interi già attraversati: Inv$\equiv(s=\sum_{i=0}^{x-1} i\wedge 0\leq x\leq n)$ e expt$=n-x$.

Conto. Ingresso: dopo `x := 0; s := 0` la somma vuota vale 0, quindi Inv vale; la guardia è sempre definita.

Invarianza: si lavora all'indietro su Inv$\wedge x<n$. Prima di `x := x + 1` serve $s=\sum_{i=0}^{x} i\wedge 0\leq x+1\leq n$. Prima di `s := s + x` serve $s+x=\sum_{i=0}^{x} i\wedge 0\leq x+1\leq n$. Ma $\sum_{i=0}^{x} i=\sum_{i=0}^{x-1} i+x$, quindi la prima congiunzione segue da Inv, e la seconda da $0\leq x<n$.

Terminazione e uscita: $n-x$ è non negativa perché $x\leq n$, e diminuisce di 1 a ogni giro. All'uscita $x\geq n$ e $x\leq n$ danno $x=n$, quindi $s=\sum_{i=0}^{n-1} i$.

Controllo. Con $n=4$, i valori di s sono $0,0,1,3,6$ e il ciclo si ferma con $x=4$: $6=0+1+2+3$. Con $n=0$ il corpo non viene eseguito e $s=0$, la somma vuota.

8.3 Un esempio: la divisione intera

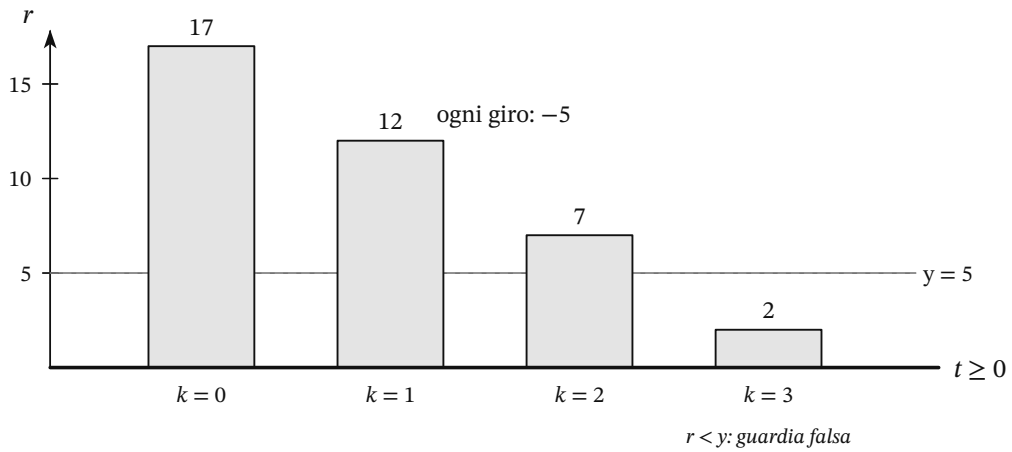
La divisione per sottrazioni del capitolo 7 soddisfa la specifica $x \geq 0 \land y > 0 \longrightarrow x = q \cdot y + r \land 0 \leq r < y$, con x, y non modificati. Verifichiamola.

ESERCIZIO GUIDATO · Divisione intera: l'invariante con $y > 0$

Idea. Ogni giro toglie y da r e aggiunge 1 a q. L'invariante è Inv$\equiv(x = q \cdot y + r \land r \geq 0 \land y > 0)$ e la funzione di terminazione expt$=r$. Il vincolo $y > 0$ è aggiunto dal libro: serve a far diminuire r.

Conto. Ingresso: dopo `q := 0; r := x` si ha $x = 0 \cdot y + x$, $r = x \geq 0$, e $y > 0$ è la precondizione. *Invarianza*: se $q' = q + 1$ e $r' = r - y$, allora $q' \cdot y + r' = (q+1)y + (r-y) = q \cdot y + r = x$; la guardia dà $r - y \geq 0$; $y > 0$ non cambia. *Terminazione e progresso*: $r \geq 0$, e $r' = r - y < r$ perché $y > 0$. *Uscita*: $r < y$, con l'invariante, dà $q = x$ div y e $r = x$ mod y.

Controllo. Con $x = 17$ e $y = 5$ la scala di r è $17, 12, 7, 2$: si esce con $q = 3$ e $r = 2$, perché $17 = 3 \cdot 5 + 2$.



La scala di r con $x = 17$, $y = 5$: ogni gradino è una sottrazione di y; si scende finché $r < y$.

Con $y = 0$ il corpo lascia r invariato e fa crescere q: la guardia resta vera per sempre. È il motivo per cui $y > 0$ è essenziale.

8.4 Programmi annotati e regole derivate

Un programma annotato rende visibili le asserzioni che servono alla prova. Le annotazioni non sostituiscono gli obblighi: li organizzano.

```
{P}
CI;
{inv: Inv} {ft: expt}
while E do
    {Inv and E}
    C
endw
{Inv and not E}
{Q}
```

Dalle regole base si ricavano tre regole derivate. Due combinano dimostrazioni sullo stesso comando:

$$\frac{\{P_1\}\ C\ \{Q_1\} \qquad \{P_2\}\ C\ \{Q_2\}}{\{P_1 \land P_2\}\ C\ \{Q_1 \land Q_2\}} \quad \text{(cong)}$$

$$\frac{\{P\}\ C\ \{Q\} \qquad \{R\}\ C\ \{Q\}}{\{P \lor R\}\ C\ \{Q\}} \quad \text{(disg)}$$

La terza serve a trasportare una proprietà attraverso un comando che non la tocca. Sia change(C) l'insieme delle variabili che compaiono come destinazione di un assegnamento in C: è un insieme di variabili *potenzialmente* modificate.

$$\frac{\text{free}(P) \cap \text{change}(C) = \varnothing \qquad \{R\}\ C\ \{Q\}}{\{P \wedge R\}\ C\ \{P \wedge Q\}} \quad \text{(invariante)}$$

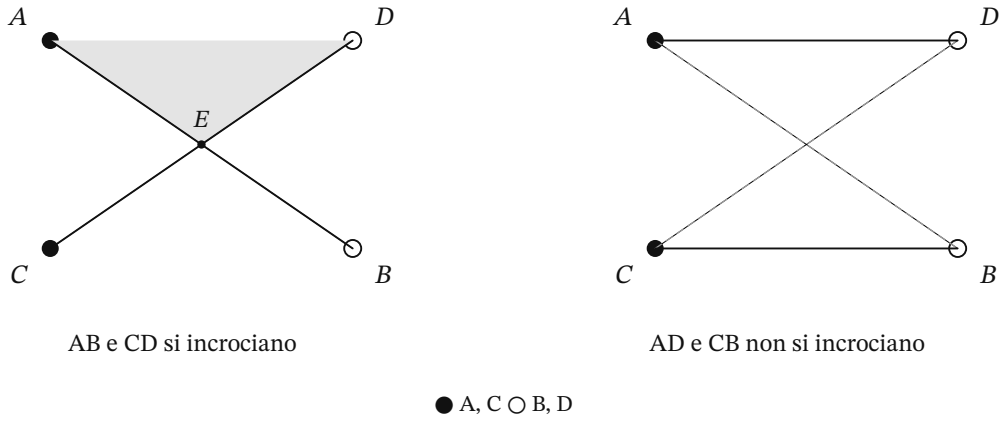
Nella divisione, il corpo assegna solo a q e r: quindi $y > 0$ si trasporta attraverso il ciclo.

8.5 Un esempio geometrico

Dati N punti blu e N punti rossi nel piano, senza tre punti allineati, si vogliono N segmenti che abbinino ogni blu a un rosso, senza che due segmenti si intersechino. Un algoritmo:

```
abbina ciascun punto blu a un diverso punto rosso;
while esistono due segmenti che si intersecano do
    sostituiscili con i due segmenti non incrociati
    che collegano ancora estremi di colore diverso
endw
```

Sia good(s) la proprietà «gli estremi di s hanno colori diversi» e good(S) = «ogni segmento di S è good». L'inizializzazione stabilisce good(S), e il corpo lo conserva. All'uscita vale quindi good(S) e nessuna coppia di segmenti si interseca: il problema è risolto, *se il ciclo termina*.



Due segmenti AB e CD che si incrociano in E vengono sostituiti da AD e CB, più corti.

Contare le intersezioni non basta: eliminarne una può crearne altre. Si sceglie invece la lunghezza totale $F(S) = \sum_{s \in S} \ell(s)$. Se AB e CD si incrociano in E, la disuguaglianza triangolare sui triangoli AED e CEB dà $\ell(AD) < \ell(AE) + \ell(ED)$ e $\ell(CB) < \ell(CE) + \ell(EB)$. Sommando, $\ell(AD) + \ell(CB) < \ell(AB) + \ell(CD)$: la lunghezza totale scende. La disuguaglianza è stretta perché non ci sono terne allineate.

Una funzione a valori reali non negativi, da sola, non prova la terminazione. Qui però le configurazioni possibili sono finite: con N punti blu e N rossi, gli abbinamenti sono $N!$. Una successione strettamente decrescente in un insieme finito non può essere infinita.

8.6 Radice cubica intera

Con $x \geq 0$ intero e non modificato, il programma è:

```
a := 0;
b := 0;
y := 0;
while y <= x do
    y := y + 3*a + 3*b + 1;
    a := a + 2*b + 1;
    b := b + 1
endw
```

Le identità $(b+1)^2 = b^2 + 2b + 1$ e $(b+1)^3 = b^3 + 3b^2 + 3b + 1$ suggeriscono di mantenere $a = b^2$ e $y = b^3$. Con $J \equiv (a = b^2 \wedge y = b^3 \wedge (b-1)^3 \leq x)$: all'inizio $a = b = y = 0$ e $(b-1)^3 = -1 \leq x$. Il corpo aggiorna y per primo, perché usa i vecchi valori di a e b; poi a, che usa il vecchio b; infine b. **L'ordine degli assegnamenti è essenziale.**

All'uscita valgono J e $y > x$, quindi $(b-1)^3 \leq x < b^3$, cioè $b - 1 = \lfloor x^{1/3} \rfloor$.

ATTENZIONE · Il programma calcola un valore in più
Il codice delle slide, da solo, lascia in b la parte intera della radice cubica *più uno*: con $x = 0$ il corpo gira una volta e $b = 1$; con $x = 8$, $b = 3$. Per avere la radice in b serve l'assegnamento `b := b - 1` dopo `endw`. Questa correzione è aggiunta al programma delle slide.

Per la terminazione (svolgimento aggiunto) si rafforza l'invariante con $b \geq 0 \wedge x \geq 0$ e si prende expt$= x + 1 - (b-1)^3$. Da $(b-1)^3 \leq x$ segue expt≥ 1. Dopo un giro, expt$' = x + 1 - b^3$, e expt$-$expt$' = b^3 - (b-1)^3 = 3b(b-1) + 1 > 0$ per ogni intero $b \geq 0$.

Traccia con $x = 10$, come (a, b, y):

giro	a	b	y	guardia $y \leq 10$
inizio	0	0	0	t
1	1	1	1	t
2	4	2	8	t
3	9	3	27	f

Il risultato è $b - 1 = 2 = \lfloor 10^{1/3} \rfloor$.

8.7 Fattoriale

Con $n \geq 0$ non modificato, si vuole $\{n \geq 0\}\ C\ \{\text{fatt}{=}n!\}$. Il programma è:

```
fatt := 1;
x := 1;
while x <= n do
    fatt := fatt * x;
    x := x + 1
endw
```

L'invariante è Inv$\equiv$(fatt$=(x-1)! \wedge 1 \leq x \leq n+1$): il limite inferiore 1 rende definito $(x-1)!$. Per la terminazione si prende expt$= n + 1 - x$. Non $n - x$: nello stato finale $x = n + 1$ darebbe -1, che viola Inv$\Rightarrow$expt≥ 0. Il ciclo termina con $x = n + 1$, quindi fatt$=n!$. Per $n = 0$ il corpo non gira e fatt$= 1 = 0!$.

8.8 Massimo comune divisore per sottrazioni

Con a, b interi positivi non modificati, si vuole $\{a > 0 \wedge b > 0\}\ C\ \{x = \text{mcd}(a, b)\}$. Il programma:

```
x := a;
y := b;
while x != y do
    if x > y
        then x := x - y
        else y := y - x
    fi
endw
```

Se all'uscita $x = y$, allora $\mathrm{mcd}(x, y) = x$. Basta quindi *conservare* il massimo comun divisore: $\mathrm{Inv} \equiv (x > 0 \land y > 0 \land \mathrm{mcd}(x, y) = \mathrm{mcd}(a, b))$. Le proprietà usate sono $\mathrm{mcd}(x, y) = \mathrm{mcd}(x - y, y)$ per $x > y > 0$ e la simmetrica.

Ogni ramo conserva Inv, e expt$= x + y$ è positiva e diminuisce: nel primo ramo il nuovo valore è $(x - y) + y = x < x + y$, nel secondo $x + (y - x) = y < x + y$. Quindi il ciclo termina con $x = y$, e $\mathrm{mcd}(a, b) = \mathrm{mcd}(x, x) = x$.

La tecnica generale è questa: si scelgono variabili di lavoro x_i, inizializzate con i dati a_i, e si conserva una quantità $F(x_1, \dots) = F(a_1, \dots)$. Nel MCD la quantità è mcd, e all'uscita la guardia falsa rende semplice estrarre il risultato.

8.9 Ricavare un assegnamento dalla quantità da conservare

Si cerca E tale che la tripla $\{x\,y + p\,q = n\}\ x := x - p;\ q := E\ \{x\,y + p\,q = n\}$ sia vera. L'aggiornamento di x sottrae $p\,y$ alla quantità $x\,y + p\,q$; per compensare, q deve crescere di y:

$$E \equiv q + y$$

Verifica all'indietro: $(x - p)y + p(q + y) = x\,y - p\,y + p\,q + p\,y = x\,y + p\,q = n$. Non ci sono divisioni né accessi ad array, quindi tutte le espressioni sono definite. Se $p = 0$ la conservazione non impone vincoli su q, e $q + y$ va bene anche in quel caso.

Officina della dimostrazione · 8. I cinque obblighi di un ciclo

Un ciclo non si dimostra con un ragionamento generale: si dimostrano cinque obblighi, uno per volta.

- Scrivi l'invariante: che cosa resta vero prima di ogni giro e alla fine?
- Scegli la funzione di terminazione: un'espressione intera, non negativa, che diminuisce a ogni giro.
- Verifica Ingresso, Invarianza, Terminazione, Progresso e Uscita, nell'ordine della tabella.
- Se l'invariante ti sembra troppo debole, rafforzalo con vincoli di segno o di dominio, come $y > 0$.

Verifica 8

a) Elenca i cinque obblighi di un ciclo.

b) Il ciclo `while x ≠ 0 do x := x + 1 endw` ammette una prova di correttezza totale con invariante $x \geq 0$? Quale obbligo fallisce?

c) Per `while i < n do i := i + 1 endw`, con invariante $0 \leq i \leq n$, quale funzione di terminazione useresti e perché funziona?

Array e progetto di cicli

In breve
Un array raccoglie molti valori sotto un solo nome, ciascuno con il suo indice. Con gli array l'invariante descrive una parte già elaborata e una ancora da fare. Alla fine tre tecniche per costruire un ciclo a partire dalla postcondizione, così da non dover indovinare l'invariante.

9.1 Array

Un **array** rappresenta una sequenza finita di valori dello stesso tipo: è una funzione $v\colon [a,b) \to A$, dove $[a,b)$ sono gli indici naturali i con $a \le i < b$ e A è $\mathbb{Z}$ o Bool. Il numero di elementi è $b-a$: l'estremo b **non è un indice**. Se $a=b$ l'array è vuoto. Dominio: $\mathrm{dom}(v)=[a,b)$.

Nel linguaggio:

```
v : array [a,b) of int
w : array [a,b) of bool
```

Per esempio `v : array [0,4) of int` ha indici $0,1,2,3$. Se `v` è un array ed `E` un'espressione intera, `v[E]` è un'espressione: l'elemento di indice E. Un valore letto dall'array può servire come indice: `v[v[0]+v[1]]` è lecito. Va però verificato che l'indice sia nel dominio.

Lo stato cambia: se v è un array, $\sigma(v)$ è una funzione sugli indici, $\sigma(v)\colon \mathrm{dom}(v) \to A$. Con $\sigma(v) = \{0 \mapsto 2, 1 \mapsto 1, 2 \mapsto 10, 3 \mapsto 6\}$ e $E \equiv v[v[0]+v[1]]$:

$$\mathcal{E}(v[v[0]+v[1]],\sigma) = \sigma(v)(\sigma(v)(0)+\sigma(v)(1)) = \sigma(v)(3) = 6$$

Gli indici 0, 1 e 3 sono tutti nel dominio. Anche il valore usato come indice esterno deve essere valido.

La clausola di def per gli array è

$$\mathrm{def}(v[E]) = \mathrm{def}(E) \wedge E \in \mathrm{dom}(v)$$

quindi, per un array di dominio $[0,n)$, $\mathrm{def}(v[x+1]) \equiv 0 \le x+1 < n$.

9.2 Aggiornamento di un elemento

Un singolo elemento si modifica con `v[E] := E'`. Il comando **valuta prima, aggiorna dopo**: con $j = \mathcal{E}(E,\sigma)$ e $d = \mathcal{E}(E',\sigma)$, entrambi definiti e $j \in \mathrm{dom}(v)$, si costruisce la sequenza $w = \sigma(v)[d/j]$, cioè $w(i) = d$ se $i = j$ e $w(i) = \sigma(v)(i)$ altrimenti. Il nuovo stato associa a v la sequenza w:

$$\mathcal{C}(v[E] := E', \sigma) = \sigma[\sigma(v)[\mathcal{E}(E',\sigma)/\mathcal{E}(E,\sigma)]/v]$$

Ci sono due aggiornamenti: quello interno, dentro la sequenza, e quello esterno, sull'identificatore v.

L'assioma si scrive con $w = v[E'/E]$, la sequenza ottenuta da v ponendo E' all'indice E:

$$\{\mathrm{def}(E) \wedge \mathrm{def}(E') \wedge E \in \mathrm{dom}(v) \wedge P[w/v]\}\ v[E] := E'\ \{P\} \quad \text{(ass, array)}$$

Nell'asserzione la sostituzione riguarda l'intera sequenza: per ogni indice i, il nuovo elemento è E' se $i = E$, e il vecchio $v[i]$ altrimenti. Così si possono dimostrare postcondizioni su più posizioni.

Esercizio guidato · Somme parziali
Sia `a : array [0,n) of int` e $n > 0$. Verifica

$$\{1 \le k < n \land (\forall i \in [0,k).\ a[i] = \sum_{j=0}^{i} j)\}\ a[k] := a[k-1] + k\ \{(\forall i \in [0,k].\ a[i] = \sum_{j=0}^{i} j)\}$$

Idea. La posizione k è l'unica modificata; le posizioni $i < k$ devono restare come sono.

Conto. Definitezza: da $1 \le k < n$ seguono $k \in [0,n)$ e $k-1 \in [0,n)$, quindi la scrittura e la lettura sono lecite. *Postcondizione*: se $0 \le i < k$, l'aggiornamento non tocca i, e $a[i]$ vale ancora $\sum_{j=0}^{i} j$. Se $i = k$, il nuovo valore è $a[k-1] + k = \sum_{j=0}^{k-1} j + k = \sum_{j=0}^{k} j$.

Controllo. Con $k = 1$: $a[0] = 0$, quindi $a[1] = 0 + 1 = 1$, che è la somma $0 + 1$. Con $k = 2$: $a[2] = a[1] + 2 = 3 = 0 + 1 + 2$.

Il modello tipico delle prove sugli array: si verificano separatamente la posizione modificata e quelle che restano invariate.

9.3 Incrementare tutti gli elementi

Sia `a : array [0,n) of int` e V una variabile di specifica che rappresenta la sequenza iniziale. Vogliamo

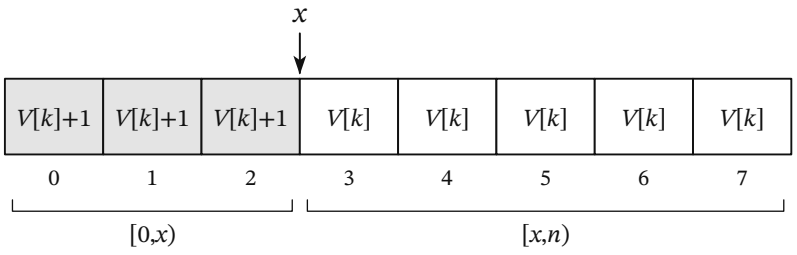
$$\{n \ge 0 \land (\forall k \in [0,n).\ a[k] = V[k])\}\ C\ \{(\forall k \in [0,n).\ a[k] = V[k] + 1)\}$$

con il programma

```
x := 0;
while x < n do
    a[x] := a[x] + 1;
    x := x + 1
endw
```

L'invariante divide l'array in due parti: quella già elaborata e quella ancora da elaborare.

$$\begin{aligned} \text{Inv} \equiv\ & 0 \le x \le n \\ & \land (\forall k \in [0,x).\ a[k] = V[k] + 1) \\ & \land (\forall k \in [x,n).\ a[k] = V[k]) \end{aligned}$$



L'array diviso in due zone: a sinistra di x i valori sono già incrementati, a destra di x sono ancora quelli iniziali V. Il corpo sposta il confine di una posizione.

La seconda parte è importante: quando si legge $a[x]$ bisogna sapere che contiene ancora il valore $V[x]$.

Svolgimento aggiunto. Con $x=0$ il prefisso elaborato è vuoto e la seconda parte coincide con la precondizione. Sotto Inv$\wedge x<n$, si ha $x\in[0,n)$, quindi l'accesso e la scrittura sono definiti. Dopo l'aggiornamento, le posizioni $k<x$ restano uguali, la posizione x diventa $V[x]+1$, e quelle $k>x$ restano $V[k]$. Dopo `x := x + 1` si ritrova Inv. La quantità $n-x$ è non negativa e diminuisce di 1. All'uscita $x\geq n$ e $x\leq n$ danno $x=n$: tutto l'array è nel prefisso elaborato.

9.4 Progettare un ciclo

Data una specifica $\{P\}\ C\ \{Q\}$, si cerca una scomposizione

$$P'\wedge G\Rightarrow Q$$

in cui P' possa essere un invariante e G sia la negazione di una guardia E esprimibile nel linguaggio: $G\equiv\neg E$. Il programma ha la forma CI; `while E do C endw`, con $C\,I$ che stabilisce l'invariante e C che lo conserva. Restano da verificare definitezza, invarianza e terminazione. È una strategia di progetto, non una ricetta che risolve ogni problema. Ecco tre tecniche.

Tecnica 1: scomporre la postcondizione. Per la divisione intera, con $x\geq 0$ e $y>0$, la postcondizione $q=x$ div y equivale a $\exists r.(x=q\cdot y+r\wedge 0\leq r<y)$. Si introduce una variabile r che sia il testimone dell'esistenziale e si separa:

$$x=q\cdot y+r\wedge r\geq 0\quad\wedge\quad r<y$$

dove la prima parte è l'invariante e $r<y$ è la parte G. La guardia è $E\equiv r\geq y$; l'inizializzazione è `q := 0; r := x`, il corpo è `q := q + 1; r := r - y`, e la funzione di terminazione è r. La dimostrazione completa è nel capitolo 8, dove si conserva anche $y>0$, necessario perché r diminuisca.

Tecnica 2: sostituire una costante con una variabile. Vogliamo $s=\sum_{i=0}^{n-1}a[i]$ con n non modificato. Si sostituisce la costante n con una variabile x nella postcondizione: $s=\sum_{i=0}^{x-1}a[i]$. Con $x=n$ si ottiene il risultato; la guardia candidata è $x\neq n$. Per avere indici validi e una funzione di terminazione naturale si rafforza l'invariante con $0\leq x\leq n$, e si prende expt=$n-x$.

```
x := 0;
s := 0;
while x != n do
    s := s + a[x];
    x := x + 1
endw
```

Il rafforzamento $0\leq x\leq n$ serve a tre cose: controllare gli indici, limitare il contatore e rendere non negativa la funzione di terminazione. Sotto $x\neq n$, si ha $0\leq x<n$, quindi $a[x]$ è definito.

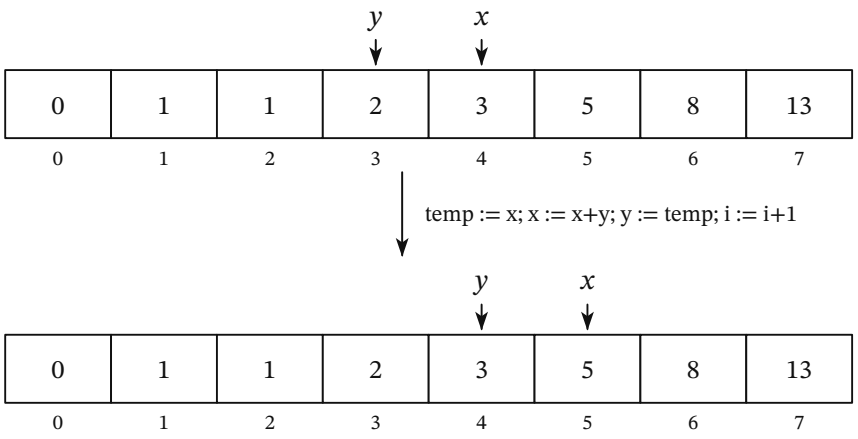
Tecnica 3: variabile ausiliaria. La successione di Fibonacci è definita da fib$(0)=0$, fib$(1)=1$ e fib$(j)=$fib$(j-1)+$fib$(j-2)$ per $j\geq 2$. Si vuole $\{k\geq 1\}\ C\ \{x=\text{fib}(k)\}$ con k non modificato.

Una prima idea è l'invariante $x=\text{fib}(i)\wedge 1\leq i\leq k$, con $E\equiv i\neq k$ e expt=$k-i$. Ma per avanzare servirebbe fib$(i+1)=x+$fib$(i-1)$, e fib$(i-1)$ non è un'espressione del linguaggio. Si introduce quindi y che ne conservi il valore:

$$\text{Inv}\equiv x=\text{fib}(i)\wedge y=\text{fib}(i-1)\wedge 1\leq i\leq k$$

Il desiderato è $x'=x+y$ e $y'=x$, in termini dei vecchi valori. La sequenza `x := x + y; y := x` non va bene, perché il secondo comando legge il nuovo x. Serve una variabile ausiliaria:

```
temp := x;
x := x + y;
y := temp
```



Dopo `temp := x` il vecchio valore di x è salvo. Il passo successivo calcola il termine seguente, e `y := temp` sposta il vecchio x nella coppia.

Il programma completo è:

```
x := 1;
i := 1;
y := 0;
while i != k do
    temp := x;
    x := x + y;
    y := temp;
    i := i + 1
endw
```

Con expt$=k-i$: all'inizio fib$(1)=1$ e fib$(0)=0$; se $i<k$, il corpo produce la coppia successiva e porta $i+1\le k$; expt è non negativa e diminuisce di 1. All'uscita $i=k$, quindi $x=\text{fib}(k)$. Per $k=1$ il ciclo non gira; il caso $k=0$ è escluso dalla specifica.

9.5 Bandiera a due colori

Sia `a : array [0,n) of int`. Vogliamo permutare gli elementi in modo che prima vengano i valori *negativi* e poi quelli *positivi o nulli*. Sia A la sequenza iniziale, e perm(a, A) la relazione: a e A hanno gli stessi elementi con le stesse molteplicità, in qualunque ordine.

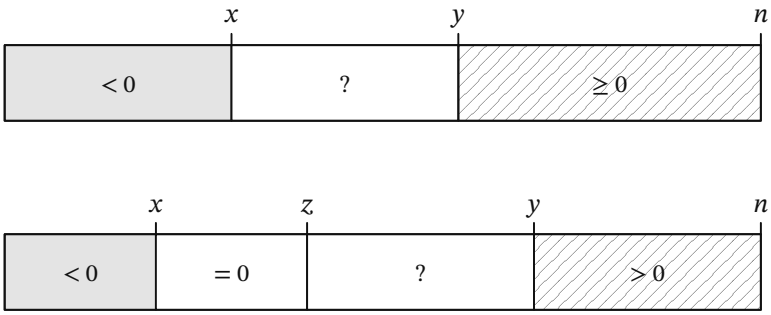
$$\{n\ge 0\wedge a=A\}\ C\ \{\text{perm}(a,A)\wedge 0\le x\le n\wedge(\forall i\in[0,x).\,a[i]<0)\wedge(\forall i\in[x,n).\,a[i]\ge 0)\}$$

Si introducono due confini x e y, con invariante

$$\begin{aligned}\text{Inv}\equiv&\text{perm}(a,A)\wedge 0\le x\le y\le n\\ &\wedge(\forall i\in[0,x).\,a[i]<0)\\ &\wedge(\forall i\in[y,n).\,a[i]\ge 0)\end{aligned}$$

Intervallo	Che cosa sappiamo
$[0,x)$	tutti gli elementi sono negativi
$[x,y)$	gli elementi sono ancora da classificare
$[y,n)$	tutti gli elementi sono non negativi

La guardia è $x\neq y$ e la funzione di terminazione è $y-x$, la lunghezza della zona non classificata. L'inizializzazione `x := 0; y := n` rende vuote le zone già classificate.



La bandiera a due colori: il prefisso negativo cresce da sinistra, il suffisso non negativo da destra. Nel mezzo resta la zona ancora da classificare, che si restringe a ogni giro.

Sotto Inv$\wedge x \neq y$ si ha $x < y$, quindi x e $y-1$ sono indici validi.

- Se $a[x] < 0$, basta incrementare x: l'elemento entra nel prefisso negativo.
- Se $a[x] \geq 0$, va portato a destra. Si scambia $a[x]$ con $a[y-1]$ e si decrementa y. x *non* avanza: il valore arrivato da destra deve ancora essere classificato. Una semplice copia `a[y-1] := a[x]` perderebbe il vecchio elemento di destra; lo scambio conserva $\mathrm{perm}(a, A)$.

```
x := 0;
y := n;
while x != y do
    if a[x] < 0
        then x := x + 1
        else temp := a[y-1];
             a[y-1] := a[x];
             a[x] := temp;
             y := y - 1
    fi
endw
```

In entrambi i rami l'invariante è conservato e $y - x$ diminuisce di 1. Lo scambio è corretto anche quando $x = y - 1$: scambia una posizione con sé stessa. All'uscita $x = y$ e la zona sconosciuta è vuota.

ATTENZIONE · Correzioni alle slide
Nella slide 395 i segni dei due casi sono invertiti: i casi coerenti con l'invariante e con il codice della slide 396 sono $a[x] < 0$ e $a[x] \geq 0$. Nella slide 397 l'ultimo quantificatore riguarda $a[i]$, non $a[y]$, e il confine finale vale $x \in [0, n]$, non $[0, n)$: $x = n$ è possibile se tutti gli elementi sono negativi, e $n = 0$ deve essere ammesso.

9.6 Bandiera a tre colori

Svolgimento aggiunto. La slide 398 chiede di estendere la partizione a tre categorie: negativi, zeri, positivi. Vogliamo due confini finali $x \leq z$ e la postcondizione

$$\begin{aligned}&\mathrm{perm}(a, A) \wedge 0 \leq x \leq z \leq n\\ &\wedge (\forall i \in [0, x).\, a[i] < 0)\\ &\wedge (\forall i \in [x, z).\, a[i] = 0)\\ &\wedge (\forall i \in [z, n).\, a[i] > 0)\end{aligned}$$

Si usa un terzo confine y, con invariante

$$\begin{aligned}&\text{Inv}\equiv \text{perm}(a,A)\wedge 0\le x\le z\le y\le n\\ &\wedge(\forall i\in[0,x).\, a[i]<0)\\ &\wedge(\forall i\in[x,z).\, a[i]=0)\\ &\wedge(\forall i\in[y,n).\, a[i]>0)\end{aligned}$$

La parte sconosciuta è $[z,y)$; la guardia è $z\neq y$, la funzione di terminazione $y-z$, l'inizializzazione $x=z=0,\ y=n$.

```
x := 0;
z := 0;
y := n;
while z != y do
    if a[z] < 0
        then temp := a[x];
             a[x] := a[z];
             a[z] := temp;
             x := x + 1;
             z := z + 1
        else if a[z] = 0
                 then z := z + 1
                 else y := y - 1;
                      temp := a[y];
                      a[y] := a[z];
                      a[z] := temp
             fi
    fi
endw
```

Se $a[z]<0$, lo scambio porta il negativo all'inizio della zona degli zeri. Quando $x<z$, il vecchio $a[x]$ è zero e passa in posizione z; quando $x=z$, lo scambio non cambia nulla. Se $a[z]=0$, la zona degli zeri si allarga. Se $a[z]>0$, si restringe da destra la zona sconosciuta e si porta il positivo nella posizione appena aggiunta al suffisso; z resta fermo per riesaminare il valore arrivato da destra. Sotto $z<y$, tutti gli indici sono validi, perché il nuovo y è il vecchio $y-1\ge z$. Ogni ramo diminuisce $y-z$ di 1; all'uscita $z=y$ e la postcondizione vale.

9.7 Una successione che dimezza l'indice

Consideriamo la successione

$$f(0)=0,\quad f(1)=1$$

Per $j\ge 2$ pari si pone $f(j)=f(j/2)$; per $j\ge 2$ dispari, $f(j)=f((j-1)/2)+f((j+1)/2)$. La specifica è $\{n\ge 0\}\ C\ \{h=f(n)\}$, con n non modificato. Far crescere un indice da 0 a n, come per Fibonacci, obbliga a conservare valori a indici dimezzati, e la preservazione dell'invariante diventa difficile. Conviene seguire la struttura della ricorsione e *far diminuire l'indice*.

Ecco $f(19)$:

$$\begin{aligned}& f(19)\\ =\ & f(9)+f(10) && \{19 \text{ dispari, } 10 \text{ pari}\}\\ =\ & f(4)+2f(5) && \{f(10)=f(5),\ f(9)=f(4)+f(5)\}\\ =\ & 3f(2)+2f(3) && \{f(4)=f(2),\ f(5)=f(2)+f(3)\}\\ =\ & 5f(1)+2f(2) && \{f(3)=f(1)+f(2)\}\\ =\ & 7 && \{f(1)=f(2)=1\}\end{aligned}$$

Ogni espressione intermedia è una combinazione di due termini consecutivi $f(x)$ e $f(x+1)$. Suggerisce l'invariante

$$\text{Inv}\equiv 0\le x\le n\wedge f(n)=k\,f(x)+h\,f(x+1)$$

dove k è una variabile di lavoro, distinta dal parametro k di Fibonacci. L'inizializzazione `x := n; k := 1; h := 0` lo stabilisce, perché $f(n) = 1 \cdot f(n) + 0 \cdot f(n+1)$. La guardia è $x \neq 0$. All'uscita $f(n) = k \cdot f(0) + h \cdot f(1) = h$, e il risultato è già in h.

Aggiornamenti per parità. Sia $q = x \operatorname{div} 2$.

- *Caso pari*, $x = 2q \geq 2$. Allora $f(x) = f(q)$ e $f(x+1) = f(q) + f(q+1)$, quindi $kf(x) + hf(x+1) = (k+h)f(q) + hf(q+1)$. Si esegue `k := h + k` e poi `x := x div 2`.
- *Caso dispari*, $x = 2q+1$ (svolgimento aggiunto). Allora $f(x) = f(q) + f(q+1)$ e $f(x+1) = f(q+1)$, quindi $kf(x) + hf(x+1) = kf(q) + (k+h)f(q+1)$. Si esegue `h := h + k` e poi `x := x div 2`. L'identità vale anche per $x = 1$, perché $q = 0$ e $f(1) = f(0) + f(1)$.

```
x := n;
k := 1;
h := 0;
while x != 0 do
    if x mod 2 = 0
        then k := h + k
        else h := h + k
    fi;
    x := x div 2
endw
```

Per la terminazione si prende expt=x. Sotto l'invariante $x \geq 0$, e la guardia dà $x > 0$, quindi $0 \leq x \operatorname{div} 2 < x$. Le divisioni e il modulo hanno divisore costante $2 \neq 0$, quindi sono definiti. Per $n = 0$ il ciclo non gira e $h = 0 = f(0)$.

Traccia con $n = 19$, come (x, k, h):

giro	x	k	h	caso
inizio	19	1	0	—
1	9	1	1	dispari
2	4	1	2	dispari
3	2	3	2	pari
4	1	5	2	pari
5	0	5	7	dispari

Il risultato è $h = 7 = f(19)$.

OFFICINA DELLA DIMOSTRAZIONE · 9. Tre modi per trovare l'invariante

Ogni tecnica di questo capitolo risponde a una domanda diversa.

- *Scomporre*: se la postcondizione è un'esistenza o una congiunzione, togli una parte e falla diventare la guardia negata ($r < y$).
- *Costante in variabile*: se la postcondizione contiene un numero fisso, sostituiscilo con una variabile che va da 0 a quel numero, e rafforza con $0 \leq x \leq n$.
- *Variabile ausiliaria*: se un aggiornamento ha bisogno di un valore che il linguaggio non sa esprimere, conservalo in una variabile in più; ricordati `temp`.

Per gli array, dividi il dominio in zone e scrivi che cosa sai di ciascuna.

VERIFICA 9

a) Nel programma di Fibonacci, perché serve `temp`?

b) Nella bandiera a due colori, perché x non avanza nel ramo $a[x] \geq 0$?

c) Calcola $f(6)$ con la definizione.

Appendice A

La cassetta degli attrezzi

Le dimostrazioni di questo libro usano pochi schemi, sempre gli stessi. Quando non sai da dove cominciare, guarda la forma della frase da dimostrare e cerca qui l'attrezzo adatto.

Se la frase dice...	**prova a...**	**vedi**
«ciò che il sistema deve fare»	scrivi lo stato iniziale e quello finale, e cerca le parole vaghe	officina 1, cap. 1
«basta un caso?», «per ogni»	cerca un controesempio piccolo; se non c'è, dimostra per tutti i casi	officina 2, cap. 2
«dimostra che...»	scrivi una catena, un passo per riga, ogni passo con la sua legge	officina 3, cap. 3
«se e solo se», «ogni ... è ...»	traduci una proposizione alla volta, e controlla le parentesi	officina 4, cap. 4
«per ogni x esiste un y»	scrivi l'ordine dei quantificatori; cerca un controesempio con un dominio piccolo	officina 5, cap. 5
«che cosa fa questo programma?»	eseguilo a mano su un caso piccolo, con una colonna per variabile	officina 6, cap. 6
«calcola la precondizione»	parti dalla postcondizione e risali comando per comando, con (ass)	officina 7, cap. 7
«il ciclo termina e fa...»	scrivi invariante, guardia e funzione di terminazione, poi verifica i cinque obblighi	officina 8, cap. 8
«per ogni elemento dell'array»	separa le zone (già elaborata, da elaborare) e scrivi che cosa sai di ciascuna	officina 9, cap. 9

Un'abitudine vale per tutti: prima di dimostrare, prova su un caso piccolo e sui valori limite. Una traccia sbagliata fa emergere l'errore in pochi minuti.

Risposte alle verifiche

1.

a) Viola la non ambiguità: «dolce» non è una condizione che un esecutore possa controllare (quando è «dolce»?). Forse viola anche la finitezza, se si aggiunge zucchero senza fine.

b) $\{x \mapsto 3, y \mapsto 3\}$ è uno stato. $\{x \mapsto 3, x \mapsto 4\}$ non lo è: uno stato associa a ogni identificatore al più un valore, e qui x ne avrebbe due.

c) Bastano dieci confronti: ogni confronto dimezza i casi possibili, e $2^{10} = 1024 \geq 1000$.

2.

a) $e \Rightarrow \neg p$. «Esco solo se non piove» dice che uscire richiede non piovere: $\neg p$ è condizione necessaria per e.

b) La formula è soddisfacibile, perché basta $p = \mathsf{f}$. Non è una tautologia: con $p = \mathsf{t}$ e $q = \mathsf{f}$ diventa falsa.

c) Con $a = \mathsf{f}$ e $b = \mathsf{t}$ le premesse sono vere e la conclusione $\neg b$ è falsa: il controesempio mostra che l'argomento non è valido.

3.

a) In $\neg(P \Rightarrow Q) \wedge R$ la P è positiva (attraversa due negazioni: quella esterna e l'antecedente di $\Rightarrow$), Q è negativa (una sola negazione), R è positiva. Infatti $\neg(P \Rightarrow Q) \wedge R \equiv P \wedge \neg Q \wedge R$.

b) $P \wedge (\neg P \vee Q) \equiv (P \wedge \neg P) \vee (P \wedge Q)$ per distributività, $\equiv F \vee (P \wedge Q)$ per contraddizione, $\equiv P \wedge Q$ per unità.

c) $\neg P \equiv P \uparrow P$, perché $P \uparrow P \equiv \neg(P \wedge P) \equiv \neg P$.

4.

a) Sono libere la y in $q(x, y)$ e la x in $r(x)$. La x in $q(x, y)$ è legata dal $\forall x$ interno; quella in $p(x)$ dal $\exists x$.

b) $\forall x.\text{pari}(x) \Rightarrow \text{dispari}(\text{succ}(x))$ è una formula; $\text{succ}(x)$ è un termine.

c) In $\mathcal{I}_2$ il dominio contiene solo $5, 10, 15$, tutti multipli di 5; in $\mathcal{I}_3$ c'è anche 7, che non lo è. Il dominio fa la differenza.

5.

a) $\exists x.(p(x) \wedge \forall y.\neg q(x, y))$.

b) Se d comparisse nelle premesse, potremmo usare fatti su un elemento particolare: da $p(a)$ concluderemmo $\forall x.p(x)$, che è un'inferenza sbagliata.

c) $\forall x.\exists y.P(x, y)$ è vera: per a si usa b, per b si usa a. $\exists y.\forall x.P(x, y)$ è falsa: nessuna y va bene per entrambi.

6.

a) Lo stato finale è $\{x \mapsto 7, y \mapsto 3\}$: $x = 3 + 7 = 10$, poi $y = 10 - 7 = 3$, poi $x = 10 - 3 = 7$. È lo scambio senza variabile ausiliaria.

b) La x compare nella postcondizione ed è modificata: $x = 2 \cdot x$ nello stato finale vale solo per $x = 0$. Serve una variabile di specifica: $x = A \wedge A \geq 0 \longrightarrow x = 2 \cdot A$.

c) No: la successione è $5, 3, 1, -1, -3, \ldots$ e non passa mai per 0, quindi la guardia resta vera per sempre.

7.

a) Con $x = 5$, `x := x + 1` dà $x = 6$, non $x = 5$: la tripla non è soddisfatta.

b) Con (ass): $\mathrm{def}(x \cdot 2) \wedge (y > 10)[x \cdot 2 / y] \equiv (x \cdot 2 > 10) \equiv (x > 5)$. La precondizione richiesta è $x > 5$.

c) Con $x = 2$, $1 \operatorname{div} 2 = 0 \neq 1$: la postcondizione non vale. Con $x = 0$ la divisione non è definita. La tripla non è soddisfatta.

8.

a) Ingresso, invarianza, terminazione, progresso e uscita.

b) No, con l'invariante $x \geq 0$. Da $x = 1$ la guardia resta vera per sempre, quindi nessuna funzione di terminazione può esistere. L'obbligo che fallisce è il progresso: nessuna quantità intera non negativa diminuisce a ogni giro.

c) Si usa expt=$n - i$. Sotto l'invariante $0 \leq i \leq n$, $n - i \geq 0$: è il terzo obbligo. Se $i < n$, allora $n - (i + 1) < n - i$: è il progresso. Il ciclo si ferma con $i = n$.

9.

a) Il secondo comando, `y := x`, leggerebbe il nuovo x: il vecchio valore va salvato in `temp` prima di aggiornare x.

b) L'elemento arrivato da destra, $a[y - 1]$, non è ancora classificato: va esaminato di nuovo, quindi x resta fermo.

c) $f(6) = f(3) = f(1) + f(2) = 1 + 1 = 2$.